KNOWLEDGE BASED SYSTEMS

A MediQuant Company

KB_SQL DATA DICTIONARY GUIDE

A guide to Data Dictionary Management

Copyright © 2023 by Knowledge Based Systems, Inc., a MediQuant Company (KBS). Printed in the United States of America.

KB_SQL ® is the registered trademark of Knowledge Based Systems, Inc., a MediQuant Company (KBS).

MUMPS is a registered trademark of the Massachusetts General Hospital.

All other trademarks or registered trademarks are properties of their respective companies.

WARNING

No part of this manual may be reproduced in any form or by any means (including electronic storage and retrieval or translation into a foreign language) without prior agreement and written consent from KBS, Inc., as governed by United States and international copyright laws.

The information contained in this document is subject to change without notice. KBS, Inc., does not warrant that this document is free of errors. If you find any problems in the documentation, please report them to us in writing.

Knowledge Based Systems, Inc., a MediQuant Company (KBS)
6200 Oak Tree Blvd, STE 150
Independence, OH 44131

Email: kbssupport@mediquant.com

Table of Contents

Preface **4**
Purpose 4
Audience 4
Syntax Conventions 4
Style Conventions 5
The Organization of this Manual 6
Chapter 1: The KB_SQL Data Dictionary **7**
The Data Dictionary Tables 7
Schemas 8
Tables, Columns, and Primary Keys 8
Data Types 9
Domains and Output Formats 9
Primary Keys and Foreign Keys 10
Index Tables 11
Key Formats 11
Summary 12
Chapter 2: Global Mapping Strategies **12**
Translating M Globals into Tables 13
Chapter 3: Creating your Data Dictionary **15**
DOMAIN EDIT Option 16
KEY FORMAT EDIT option 21
OUTPUT FORMAT EDIT option 23
SCHEMA EDIT Option 26
MAP EXISTING GLOBALS option 27
REPORTS Option 51
Chapter 4: Table Filers **53**
Overview 53
SQL Statements 55
Automatic Table Filers 57
Manual Table Filers 59
Option For Creating Table Filers 67
Chapter 5: The DDL Interface **68**

The Import DDL Interface....68
Operation....70
DDL Commands....77
Syntactical Components....80
The Export DDL Interface....83

Preface

Purpose

The purpose of the KB_SQL Data Dictionary Guide is to explain relational tables and the process of mapping M globals to a data dictionary. The data dictionary is needed by KB_SQL to retrieve data from your M database. This manual also provides information about a new technology used to update M globals as well as an alternative mapping process for these globals.

Audience

This manual is written for the technical resource who is responsible for the overall management of the KB_SQL system.

We expect you to be familiar with M, the relational database model, and SQL. For those who want to increase their understanding of these topics, we have provided a list of publications in the "Additional Documentation" section in the preface of the KB_SQL Database Administrator's Guide.

We also suggest that you review Lesson 1: The Basics in the KB_SQL SQL Reference Guide to become familiar with the functions of the interface.

**** The software versions and images shown in this document may vary from the actual version that is being installed.**

Syntax Conventions

This manual uses the following syntax conventions when explaining the syntax of a KB_SQL statement.

Feature	Example	Meaning
KEY WORDS	SELECT	An SQL keyword that should be entered exactly as shown. (However, it is not necessary for you to capitalize key words. We do so for identification purposes only.)
lowercase word	table	A language element; substitute a value of the appropriate element type.

Confidential | ©2023

or	table or view	A choice; enter either the item to the left or to the right of the *or*, but not both. If the *or* is on a separate line, enter either the line(s) above or the lines(s) below.
|	LEFT|RIGHT|CENTER	A choice; enter one of the items separated by a vertical bar.
{ }	column {,column}	The items within the braces form a required composite item. Do not enter the braces.
[ ]	table [AS alias]	The item(s) within the brackets form an optional composite item. Including this item may change the
. . .	column [,column]...	An ellipsis indicates that the item which precedes the ellipsis may be repeated one or more times.
(a) *or* (b) *or* (c)	ASCII(c)	Character literals composed of one or more characters enclosed within quotes (e.g., 'abc').
(m) *or* (n)	CHAR(n)	Numeric literals (e.g., 123 or 1.23)

Style Conventions

To help you locate and identify material easily, KB Systems uses the following style conventions throughout this manual.

[key]

Key names appear enclosed in square brackets.
Example: To save the information you entered, type Y and press [enter].

{compile-time variables}

References to compile-time replacement variables are enclosed in curly braces. The names are case sensitive.
Example: {BASE}

Italics

Italics are used to reference prompt names (entry fields) and terms that may be new to you. All notes are placed in italics.
Example: The *primary key* of the table is defined as the set of columns that is required to retrieve a single row from the table.

Window

The manual includes many illustrations of windows. Window names are highlighted by a double underline.

Prompt: data type (length) [key]

The manual includes information about all of the system prompts. Each prompt will include the data type, length, and any special keys allowed.

If the prompt is followed by a colon (Prompt:), you may enter a value for the prompt.

If a prompt is followed by an equal sign (Prompt=), it is for display purposes only.

If the prompt is followed by a question mark (Prompt?), you can enter a value of YES or NO.

^GLOBAL

All M global names will be prefixed by the ^ character.

Tag^Routine

All M routine references appear as a tag^routines.

Menu Option/Menu Option/Menu Option

A string of options shows you the sequence in which you must select the options in order to arrive at a certain function. Each menu's option is separated by a slash (/).
Example: DATA DICTIONARY/REPORTS/SCHEMA PRINT

The Organization of this Manual

The *KB_SQL Data Dictionary Guide* first discusses the internal design of the KB_SQL data dictionary and then points up some strategies for mapping your M globals to a data dictionary. Next, the manual walks you through each of the menu options available for the mapping process. An important new concept, table filers, is introduced along with a look at the process by which this technology updates globals. Lastly, the KB_SQL Data Definition Language interface is presented as an alternative for mapping existing globals.

Chapter 1: The KB_SQL Data Dictionary: Describes the internal database design of the data dictionary.

Chapter 2: Global Mapping Strategies: Suggests strategies for mapping your M globals to relational tables.

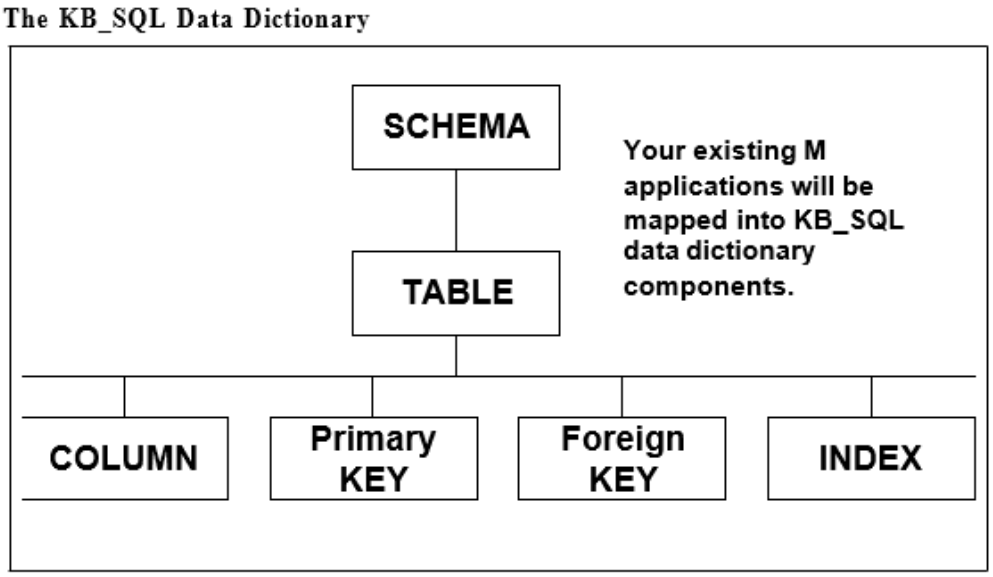
Chapter 3: Creating Your Data Dictionary: Describes each Data Dictionary option used in the global mapping process.

Chapter 4: Table Filers: Describes the table filer technology that is used to apply changes to your database.

Chapter 5: The DDL Interface: Describes an alternative for mapping existing globals.

Chapter 1: The KB_SQL Data Dictionary

This chapter discusses the components of the KB_SQL relational data dictionary. An understanding of the structure of the data dictionary will prepare you for the global mapping process after which your users will have direct, controlled access to the information resources of your organization.

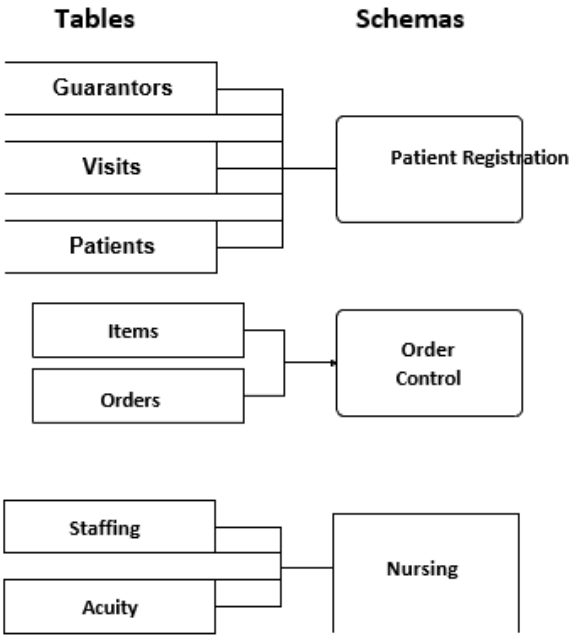


The Data Dictionary Tables

The DATA_DICTIONARY schema contains the fundamental tables for KB_SQL. For details about individual tables, use the TABLE PRINT procedure (DATA DICTIONARY/REPORTS/TABLE PRINT).

Schemas

The schema provides a high-level organization to groups of related tables. You may choose to define a schema for each of your applications (as shown by the diagram below), and then define additional schemas to provide a more detailed separation of tables.

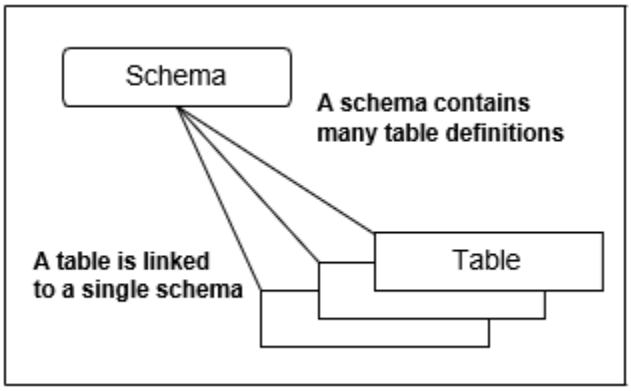


A single table may be linked to only one schema. Some tables may be accessed by multiple applications. This situation can be managed by defining a general, or central, schema to include those tables that are referenced by several applications.

Think of the schema as a logical name for a set of related tables— an organizational tool to be used in a way that best meets the needs of your clients.

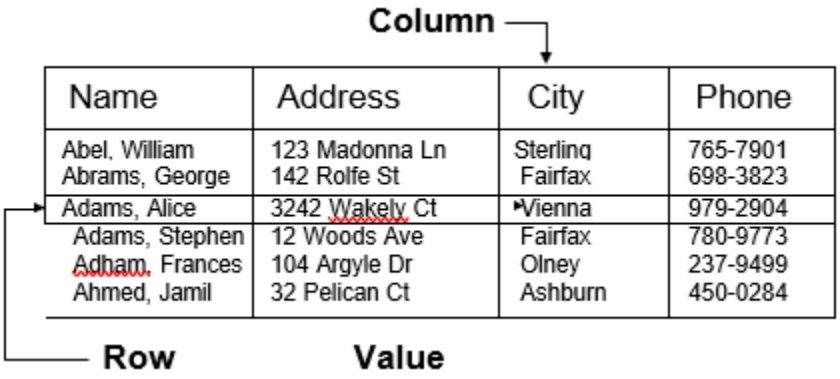
Tables, Columns, and Primary Keys

A relational database consists of a collection of *tables*. All data in the database is stored in one of these tables.



Rows, Columns, and Values

Conceptually, each table is a simple two-dimensional structure, made up of some number of *rows* and *columns*. Each column in a table is assigned a unique name and contains a particular type of data such as characters or numbers. Each row contains a value for each of the columns in the table. The intersection of the columns with each row defines the *values* in the rows.



Data Types

KB_SQL supports several data types. These data type definitions cannot be modified, nor can additional data types be added to the system. A list of the data types is provided below.

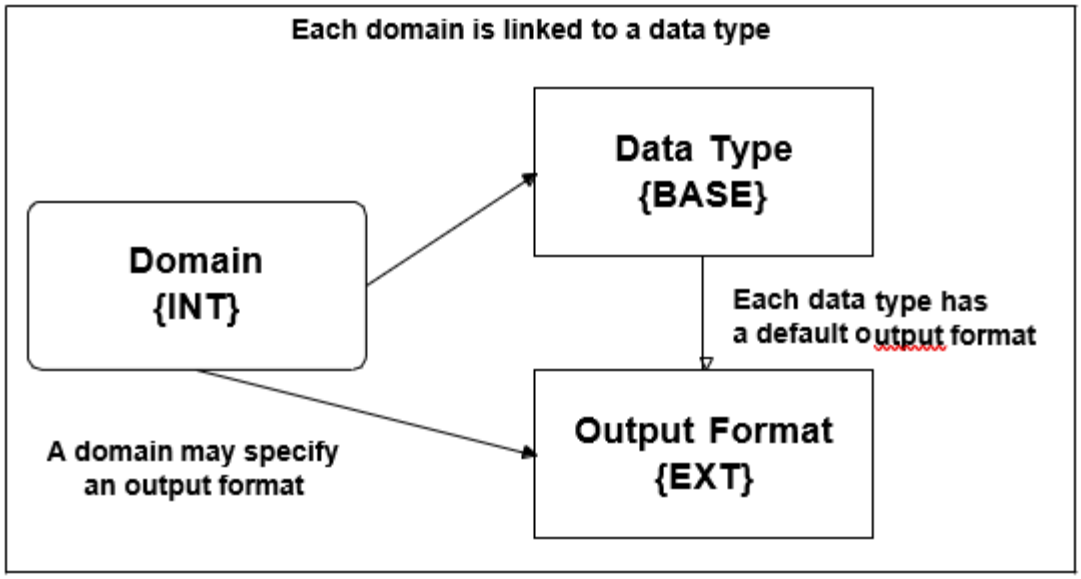
Name	Length	Format
CHARACTER	20	any characters
DATE	11	$P($H,",",1)
FLAG(yes/no)	1	1 or NULL
INTEGER	10	positive or negative digits, 999999999
MOMENT	17	$H
NUMERIC	10,2	positive or negative numbers, 9999999.99
TIME	10	$P($H,",",2)

KB_SQL knows how to compare, manipulate, and display standard data type values only. If you have data stored in another way, you must define a *domain*. The domain must specify how to transform the stored value into the base value. A discussion of domains follows.

Domains and Output Formats

If data is stored in your system in a way that cannot be expressed using one of the default data types, you must define a domain to describe the stored format. A domain can also be useful when several columns have the same data type and output format. Instead of linking each column to both a data type and an output format, the column can be linked to a single domain definition. This has an additional benefit: if a change must be made to either the domain or to the output format, all columns are changed at the same time.

Confidential | ©2023



Each data type provided by KB_SQL has a default storage format, called the {BASE} format. Every domain, supplied by either KB_SQL or the client, will have an internal {INT} format. If the internal format of the domain is different from the base format of the data type, conversion logic must be specified to transform the value in both directions.

A domain may also have an output format. If not specified, the domain uses the default output format specified for the data type.

Primary Keys and Foreign Keys

The *primary key* of a table is the set of one or more columns that is unique for each row. In some cases, the key is a computer-generated internal number. In others, some unique external value can be used as the key. In any case, no part of the primary key can ever be empty or NULL.

When the complete primary key value from one table is stored as columns in another table, these columns can be used as a *foreign key*. Many queries can take advantage of foreign key to primary key joins. Notice how the join between a column in the PATIENTS table is explicitly joined with the primary key of the DOCTORS table in the following query.

Foreign key - explicit join

```
-- list patients and their primary physicians
select PATIENTS.NAME, DOCTORS.NAME
from PATIENTS, DOCTORS
where PATIENTS.PRIMARY_DOCTOR = DOCTORS.DOCTOR_ID
```

In addition to supporting this type of join, KB_SQL provides a method to perform an implicit join using a foreign key definition.

On the following page, note the special syntax used to indicate the use of a foreign key.

Essentially, the statement below says: "Use the foreign key named PRIMARY_DOCTOR_LINK to access a

particular row of information from the DOCTORS table, and return the doctor name."

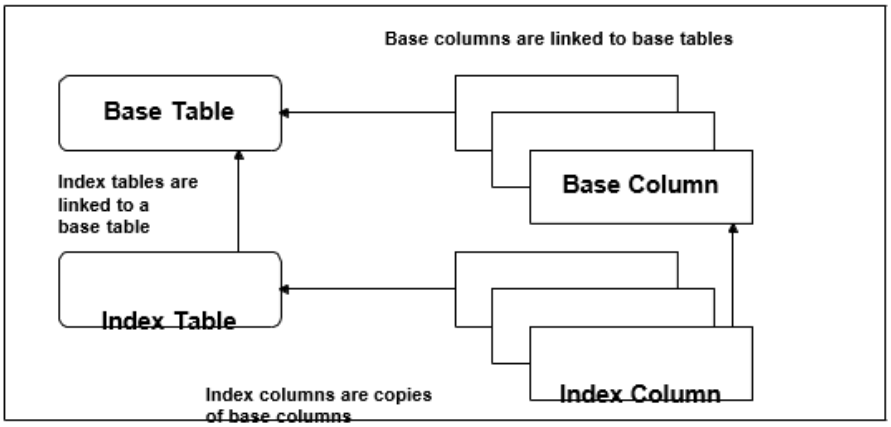
Foreign key - implicit join

```
-- list patients and their primary physicians
select NAME, PRIMARY_DOCTOR_LINK@NAME
from PATIENTS
```

Index Tables

An *index* is a physical structure that includes one or more columns from a base table. It is an alternate way to access rows in a table. The index is typically organized in a manner that provides efficient access by one or more data values as the primary keys of the index. In KB_SQL, an index is defined in the same way as a table. Any operation that can be performed on a table can also be performed on an index.

Indexes are often densely packed M (MUMPS) globals, having many more rows per physical block than the corresponding base table. This information is used by the query planner when deciding on an efficient access strategy for your queries. The diagram below shows how columns from a base table are used in an index table.



For example, an index table on patient names would include the name column from the base table. The index, PATIENT_BY_NAME, would be linked to the base table, PATIENTS. The index column, NAME, would be linked to the NAME column in the base table.

Key Formats

A *key format* is a named collection of M code that converts a value from a base table into a different format for storage in an index table. When specifying the primary keys of an index, you may optionally specify a key format. If none is specified, the key is stored in the same format as in the base table.

Column	{BASE}	{INT}	Key Format
BIRTH_DATE	43918	19610330	YYYYMMDD
		-43918	DESCENDING
		1961	YYYY

LAST_NAME	O'LEARY	OLEARY	NO_PUNCT

There are three different formats shown for date values. The internal date value is always the first part of the $H value. One format uses a YYYYMMDD format to facilitate searches by year, month, and day. The descending format indexes dates in reverse order, with more recent dates at the top of the index. The YYYY format facilitates searches by year. The YYYY format is an example of a many-to-one transformation, since a value cannot be transformed back to the base format without losing accuracy. The name format, another example of a many-to-one transformation, strips all punctuation from a base value.

Any index that has a key that is not a one-to-one transformation requires special handling. Any queries that apply constraints to the key must also test the constraints against the value in the base table.

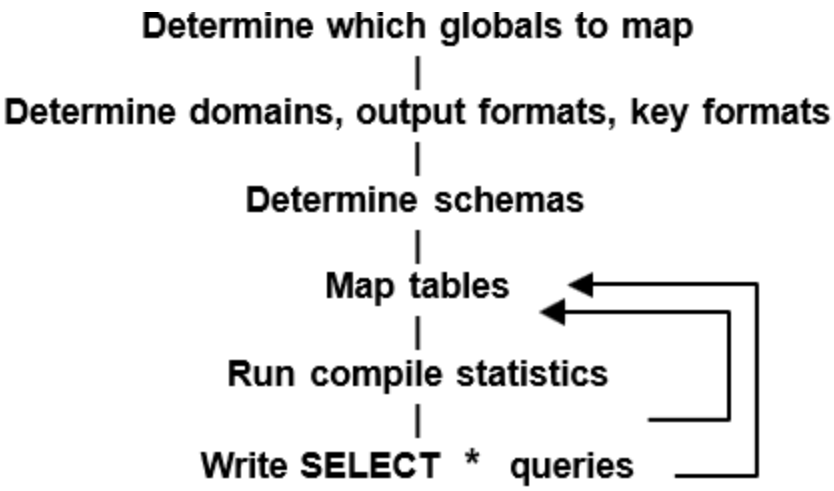
Summary

The KB_SQL data dictionary is structured to manage information on all of the tables and columns of your database. After studying the relational data dictionary model as implemented in KB_SQL, you are ready to begin the process of defining your M globals to the data dictionary.

Chapter 2: Global Mapping Strategies

The relational database model requires that all information contained in the database be presented to the users as a collection of
two-dimensional tables. The relational model does not specify rules for how the data is actually stored, only for how the data is presented. It is therefore possible to store data in M globals and still provide a relational view.

In order to furnish users with relational tables, the DBA must map the existing globals into tables in the relational data dictionary. We recommend the following strategy:



It may not be necessary to map all globals in the system, rather just those that will be most beneficial to your users. As in any significant task, the planning phase should be as complete as possible. If all schemas, tables, domains, output formats, and key formats have been defined, the global mapping

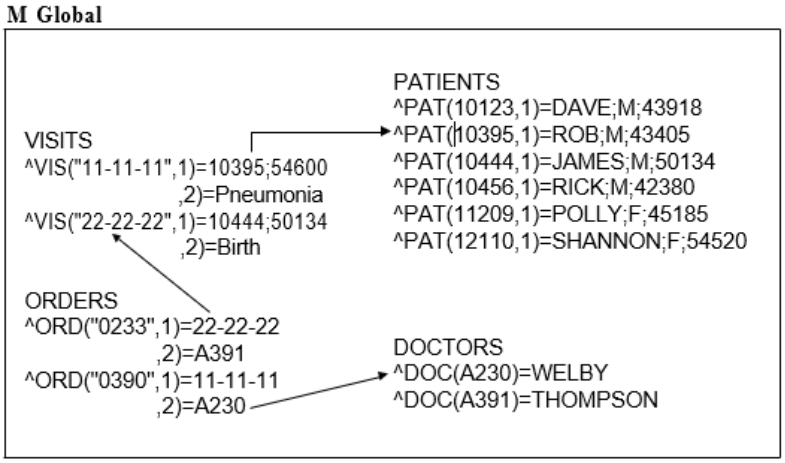
process can focus on the definition of the columns.

This section outlines various global mapping techniques, ranging from simple to more complex. We hope that these examples may benefit you and others as you undertake the global mapping process.

Translating M Globals into Tables

One major function of KB_SQL is to reference existing M globals using SQL statements. In order to accomplish this, you must translate the global definitions into corresponding relational table definitions. In many cases, this process can be partially automated, translating the information in your on-line data dictionary into the KB_SQL model. In other cases, where documentation exists only on paper, or not at all, you have more investigation to do in the analysis stage. In either case, reaching our goal requires a fundamental understanding of how KB_SQL sees globals as the internal representation of relational tables.

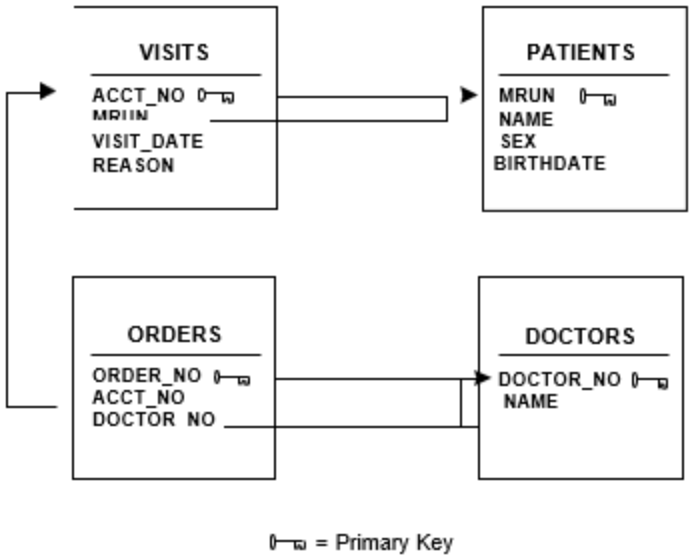
All M data is stored in persistent arrays called *globals*. The globals consist of one or more variable length keys, or *subscripts*, and a variable length data value. The absolute length of the subscripts and data depends on your M implementation. The following illustration shows how a set of four globals might be represented in a hospital application.



Relational Tables

In this simple example, the ^VIS, ^PAT, ^ORD, and ^DOC globals map into the VISITS, PATIENTS, ORDERS, and DOCTORS tables. This example is considered simple because the global subscripts are easily identified as the primary keys of the tables. If your system uses this type of global design, the mapping process will be very direct.

The lines in the illustrations are meant to show the relationships between the tables. Each line connects a foreign key from one table to the primary key of another. In the relational model, these relationships are stored as part of the database definition.



The Physical Definition

Before any data relationships can be defined, the DBA must provide the physical definition of the columns in the tables. The physical definition is comprised of a global reference and a piece reference. A column can have either or both parts. Primary keys often have just a global reference. Data columns often have just a piece reference.

^TEST(A,B,C)=D^E^F^G,H				
Column	**Parent**	**Global Reference**	**Piece Reference**	**Sample Reference**
A		^TEST(		^TEST(A
B	A	,		^TEST(A,B
C	B	,		^TEST(A,B,C
DATA	C	)		^TEST(A,B,C)
D	DATA		"^",1)	$P(DATA,"^",1)
E	DATA		"^",2)	$P(DATA,"^",2)
F	DATA		"^",3)	$P(DATA,"^",3)
GH	DATA		"^",4)	$P(DATA,"^",4)
G	GH		",",1)	$P(GH,",",1)
H	GH		",",2)	$P(GH,",",2)

The subscripts A, B, and C become columns with vertical prefixes. A data column is defined for the string of data stored at the global reference defined by the columns A, B, and C. The data columns D, E, F, G, and H are defined with horizontal suffixes for a $PIECE reference. (KB_SQL also supports a $EXTRACT reference.) Note how the column GH is used as an "artificial" column in order to simplify the definition of columns G and H. This style is often used for columns that store dates in $H format, where G is the date and H is the time component.

Primary Keys

The *primary key* of the table is the set of columns that is required to uniquely identify a single row from the table. The system can generate code to traverse each key using a standard row traversal operation. Although this situation is dominant, more complex data structures exist.

A single M data node may contain multiple occurrences of a particular data element. Since the relational model does not allow a column to contain multiple values, the DBA must define a custom primary key. KB_SQL provides several custom hooks allowing the DBA to enter the M code to traverse a complex primary key.

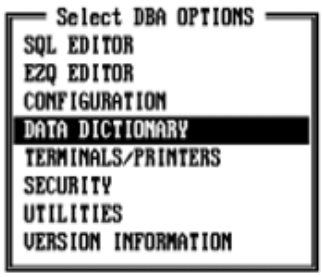
^TEST(A,B,C)=D^E^F^G1,G2,G3,...,GN				
Pkey #	**Column**	**Parent**	**Global Reference**	**Sample Looping Code**
1	A		^TEST(	S A=$O(^TEST(A))
2	B	A	,	S B=$O(^TEST(A,B))
3	C	B	,	S C=$O(^TEST(A,B,C))
4	G			S X1=$P(^TEST(A,B,C),"^",4) S X2=$L(X1,","),X3=0 S X3=X3+1 S G=$P(X1,",",X3)

The primary keys 1, 2, and 3 are simple subscripts. The system uses $ORDER to loop through these values. Primary key 4 is complex. It uses custom logic to traverse the entries in the list. Refer to the "PRIMARY KEYS Option" section in Chapter 3 for a complete description of the custom primary key logic.

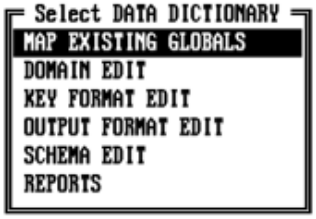
Chapter 3: Creating your Data Dictionary

The data dictionary provides a relational view of your M globals. The global mapping process is used most often and is, therefore, listed first in the menu of procedure options. However, the schemas, domains, output formats, and key formats may need to be defined before any global mapping can be completed. Therefore, this chapter discusses these options in the order in which you will use them.

Select DBA OPTIONS



Select DATA DICTIONARY



DOMAIN EDIT Option

```
Select DATA DICTIONARY
MAP EXISTING GLOBALS
DOMAIN EDIT
KEY FORMAT EDIT
OUTPUT FORMAT EDIT
SCHEMA EDIT
REPORTS
```

A *domain* provides information about how data is stored and displayed. This procedure can be used to add, edit, and delete domains. A domain should be defined for each different storage method that is used in your system.

DOMAIN Name

```
Domain Name
Domain name: ____________________________
```

Domain name : **character (30) [list]** Supply a valid SQL_IDENTIFIER or press [list] to select from a list of existing domains.

***Note**: An **SQL_IDENTIFIER** is a name, starting with a letter (A-Z), followed by letters, numbers (0-9), or underscores '_'. The last character in the name cannot be an underscore. The length of the name must not exceed 30 characters.*

If you enter the name of a new domain, the Add Domain window appears. Select YES to add a domain. Otherwise, if domains exist, you can press [list] to view the names in the selection window. From the selection window, you can press [insert] to add a new domain or select the domain you wish to edit. If you want to delete a domain, highlight it and press [delete].

ADD

Domain definitions should be added in the beginning of the global mapping process. Once a domain definition has been added, you may link columns to that domain.

EDIT

Edits to domains affect all columns that reference that domain. Be sure to determine if any columns are using the domain before editing the domain definition.

DELETE

You may not delete a domain that is referenced by a column. Otherwise, the domain can be deleted.

DOMAIN Information

Confidential | ©2023

```
Domain Information
 Domain name: DESCENDING_DATE_____________
 Description: date_____________________________________________________
   Data type: DATE_____________________________
      Length: ___ Scale: _
Output format: DD/MM/YY_____________________
      Override collating sequence? NO_
         Skip search optimization? NO_
      Skip collation optimization? NO_
              Different from base? NO_
      Reverse > and < comparisons? NO_
Perform conversion on null values? NO_
         Override data type logic? NO_
```

Domain Name: **character (30)**

The name is the logical reference for this domain. This prompt accepts a valid SQL_IDENTIFIER.

Description: **character (60)**

Provide a description for this domain.

Data Type: **character (30)**
[list]

Each domain is associated with a data type. The data type determines the {BASE} format and default output format for the domain.

Length: **integer (3)**

For character, integer, or numeric data types, the length represents the default number of characters in the data in its stored (domain internal {INT}) format.

Scale: **integer (1)**

For numeric data types, the numeric scale represents the number of digits to the right of the decimal point.

Output Format: **character (30) [list]**

Each domain may be linked to an output format. If no output format is specified, all values are formatted using the default output format for the data type.

Override Collating Sequence? **YES/NO**

The default is NO. Answer YES if the value is numeric, but the internal format collates as a string.

Skip Search Optimization? **YES/NO**

If the domain cannot be optimized, then enter YES; otherwise, enter NO. This overrides the Skip collation optimization prompt.

Note: *If KB_SQL tries to optimize a key that shouldn't be, the query returns the wrong results.*

Skip Collation Optimization? YES/NO

If the domain cannot be optimized by applying the greater than, less than, BETWEEN operators, then enter YES; otherwise, enter NO. Regardless of the answer to this prompt, the query is optimized for = and IN operators.

Different from base? YES/NO

Answer YES if the internal storage format of the domain is different from the internal format of the base type. Otherwise, the system assumes that the stored format is the same as that of the data type. When you answer YES, the *Reverse > and < comparisons* and the *Perform conversation on null values* prompts are enabled.

Reverse > and < comparisons? YES/NO

Answer Yes if the internal format is stored in a format that requires comparison operators to be reversed. (For example, a date which is stored internally as a negative number must have the reverse comparisons flag set.) After you answer YES, the Domain Logic window appears which is discussed on the next page.

Note*: If the stored format is different from the data type format, you must enter conversation code. If a numeric or integer value is store with either leading or trailing zeros, that value must be defined using a custom domain that converts it to a numeric or integer value without leading or trailing zeros. A discussion of this process begins on the next page.*

Perform conversation on null values? YES/NO

If the conversion logic must be executed for null internal values, answer YES.

Conversions

If you are adding a domain with a DATE, TIME, or MOMENT data type, you can use KB_SQL's date and time conversion routines explained in Chapter 10 of the *KB_SQL Database Administrator's Guide*.

If you answered YES to the *Different from base* prompt, the Domain Logic window appears. It contains a menu of conversion options. If the stored format is different from the data type format, you must enter conversion code. You may supply either an expression (selecting the FROM and TO EXPRESSION options) or a line of code (selecting the FROM and TO EXECUTE options).

<u>Domain Logic</u>

```
Domain Information
 Domain name: DESCENDING_DATE_____________
 Description: date_____________________________________________
   Data type: DATE_____________________________
      Length: ___ Scale: _
Output format: ____________________________
      Override collati
         Skip search o
      Skip collation o
              Differen
      Reverse > and <
Perform conversion on
         Override data
```

```
Domain Logic
FROM BASE EXPRESSION
FROM BASE EXECUTE
TO BASE EXPRESSION
TO BASE EXECUTE
*EXIT
```

The conversion code must be able to perform a two-way transformation, from domain {INT} to data type {BASE} and vice versa, without any loss of information. These conversions are necessary so that the query optimizer knows how to compare columns of data.
You may enter code in the form of an M expression, including the parameters {INT} and {BASE} to represent the internal and base formats.

The examples below illustrate how the expression and execute code can be used to accomplish the conversions. (The code is for illustrative purposes only.) Although expressions are preferred, you should use the form that is most effective for your situation. (You may want to use execute code if you can't accomplish what you need to in one expression.)

Base to Internal (Expression)

```
Convert from BASE to INT Expression
-{BASE}
```

Base to Internal (Execute)

```
Convert from BASE to INT Execute
S {INT}=-{BASE}
```

Internal to Base (Expression)

This code converts the internal to base format for use in comparisons against other date values.

```
Convert from INT to BASE Expression
-{INT}
```

Internal to Base (Execute)

```
Convert from INT to BASE Execute
S {BASE}=-{INT}
```


Confidential | ©2023

Overriding Data Type Logic

The *Override data type logic?* prompt in the Domain Information window is useful when you want to override a default conversion and data type for a particular domain. It allows you to customize your data types.

Override data type logic? **YES/NO**

Answer YES is you plan to allow special input values normally not acceptable for this domain OR if you wish to use restrictive data type validation.

The Override Data Type window appears with two menu options. Each option's corresponding drop-down window, beginning with EXTERNAL TO BASE EXECUTE, is shown in sequence on the next page.

Override Data Type

```
Domain Information
 Domain name: DESCENDING_DATE_____________
 Description: date_____________________________________________
   Data type: DATE_____________________
      Length: ___ Scale: _
Output format: ___________________________
      Override collati   Override Data Type
         Skip search o   EXTERNAL TO BASE EXECUTE
      Skip collation o   VALIDATION EXECUTE
             Differen    *EXIT
      Reverse > and <
Perform conversion on null values? NO_
         Override data type logic? YES
```

X to Base (Execute)

This code uses a site-specific routine to convert date values to $H format.

```
Convert X to Base Execute
S X=$$DATE^%ZVAL(X)
```

Validate X (Execute)

This code checks for non-negative numeric values.

```
Validate X Execute
I X<0 K X
```


Confidential | ©2023

KEY FORMAT EDIT option

```
Select DATA DICTIONARY
MAP EXISTING GLOBALS
DOMAIN EDIT
KEY FORMAT EDIT
OUTPUT FORMAT EDIT
SCHEMA EDIT
REPORTS
```

This procedure can be used by the DBA to add, edit, and delete key format definitions. Because you can store data in a data column one way and store it in the primary key a different way, you can use key format definitions to indicate how data is to be stored in the primary keys of index tables.

Key Format Name

```
Key Format Name
Key format: ____________________________
```

Key Format: **character (30) [list]**

Supply a valid SQL_IDENTIFIER or press [list] to select from a list of existing key formats.

If you enter the name of a new key format, the Add Key Format window appears. Select YES to add a key format. Otherwise, if key formats exist, you can press [list] to view them in the selection window. From the selection window, you can press [insert] to add a new key format or select the key format you wish to edit. If you want to delete a key format, highlight it and press [delete].

ADD

Key format definitions should be added in the beginning of the global mapping process. Once a key format is defined, it can be referenced by the primary keys of index tables.

EDIT

Edits to key format definitions can affect the query access planning process. Be sure to identify the index primary keys that may be affected before making any changes.

DELETE

A key format may not be deleted if it is referenced by an index table primary key. Otherwise, the key format may be deleted.

Confidential | ©2023

Key Format Information

```
Key Format Information
       Name: ________________________
Description: ____________________________________________
        One to one transform? NO_
Do all non NULL values exist? NO_
   Reverse > and < operators? NO_
```

Name: **character (30)**

The name is the logical reference for this key format definition.

Description: **character (30)**

Provide a description for this primary key format.

One to one transform? **YES/NO**

Answer YES if the key format conversion produces a distinct value for each distinct BASE value. Answer NO if the key format produces a value that may be the same for several base values. For example, if a key format converts a date into a YYMM value, the key format is not a one-to-one transform, because all dates for a particular month are converted into one YYMM value.

Do all non-NULL values exist? **YES/NO**

Answer YES if the key format will produce a non-null value for all non-null column values. Answer NO if the key format conversion can produce a null value. For example, answer NO for an index on a patient type index that only includes emergency room patients.

Reverse > and < operators? **YES/NO**

Answer YES if comparison operators must be reversed for this key format. For example, this would be relevant if a date is converted into a negative number for use in an index.

Conversions

The key format conversion is a one-way transformation of a data type {BASE} format to an internal {INT} index format. Unlike the domain conversions, it is not necessary to be able to transfer from the key format back to the internal format. As with other conversion logic, you may use either an expression or execute code. Use an expression when possible. However, you may want to use execute code if you can't accomplish what you need to in one expression.

Base to Internal (Expression)

Confidential | ©2023

```
Key Format Information
      Name: NUMERIC_STRING_____________
Description: ____________________________________________________
        One to one transform? YES
Do all non NULL values exist? NO_
   Reverse > and < operators? NO_
Convert from BASE to INT Expression
$S({BASE}?.N:{BASE}_" ",1:{BASE})
```

Base to Internal (Execute)

```
Convert from BASE to INT Execute
S {INT}=$S({BASE}?.N:" "_{BASE},1:{BASE})
```

OUTPUT FORMAT EDIT option

```
Select DATA DICTIONARY
MAP EXISTING GLOBALS
DOMAIN EDIT
KEY FORMAT EDIT
OUTPUT FORMAT EDIT
SCHEMA EDIT
REPORTS
```

The OUTPUT FORMAT EDIT option may be used to add, edit, and delete output format definitions. Output format definitions provide information about how data is displayed. KB Systems provides you with seven different data types (as shown in the Select Data Type window) and output formats for each. You may edit these formats or add additional ones.

Select Data Type

```
Select Data Type
CHARACTER
DATE
FLAG
INTEGER
MOMENT
NUMERIC
TIME
```

The following selection window lists the output formats for the CHARACTER data type.

Select, Insert, Delete Output Format

```
Select, Insert, Delete Output Format
*INTERNAL          FREE TEXT
 LOWER             lowercase
 UPPER             UPPERCASE
```

ADD

You may add an output format for every type of display value that exists in your system. Once defined, the output format can be referenced by domains.

EDIT

Changes to an output format can affect all queries that reference that output format. Be especially careful when changing the display length of the column. This can have adverse effects on column alignment in reports.

DELETE

You may not delete an output format that is referenced by either a data type or a domain. Otherwise, the output format can be deleted.

Output Format Information

```
Output Format Information
        Name: INTERNAL____________________
  Description: Character internal format_____________________________
For data type: CHARACTER___________________
     Example: FREE TEXT____________________
      Length: 60
     Justification: L
```

Name : **character (30)**

The name is the logical reference for this output format definition. The name must be a valid SQL_IDENTIFIER.

Description : **character (60)**

Enter a description for the output format.

For data type: **character (30)**

Enter the data type (e.g., character, integer, date) to which this format applies.

Example : **character (30)**

Provide an example showing the data type in this format.

Length : **character (3)**

The length is the maximum number of characters in the external {EXT} format for this output format.

Justification : **character (1) (L,R)**

The justification (left or right) for an output format. The value should either be R for right justified or L for left justified.

Conversions

The output format conversion is a one-way transformation of a data type internal {BASE} value to an external {EXT} display format.

Unlike the data type and domain conversions, it is not necessary to be able to transfer from the external back to the internal format. As with other conversion logic, you may use either an expression or execute code. (You may want to use execute code if you can't accomplish what you need to in one expression.) The logic can include the following parameters.

Compile-Time Parameter	Description
{BASE}	Data type format
{LENGTH}	External length of value
{SCALE}	Number of digits to the right of decimal point
{EXT}	External value

Base to External (Expression)

```
Output Format Information
        Name: INTERNAL
 Description: Character internal format
For data type: CHARACTER
     Example: FREE TEXT
BASE to EXT Conversion Expression
$J({BASE},"",2)
```

Base to External (Execute)

```
Output Format Information
        Name: INTERNAL____________________
 Description: Character internal format_____________________________
For data type: CHARACTER_____________________
     Example: FREE TEXT_____________________
BASE to EXT Conversion Execute
S KDI={BASE},KDF=10 D DE^SQL0DT S {EXT}=KDE
```

SCHEMA EDIT Option

```
Select DATA DICTIONARY
MAP EXISTING GLOBALS
DOMAIN EDIT
KEY FORMAT EDIT
OUTPUT FORMAT EDIT
SCHEMA EDIT
REPORTS
```

This procedure can be used by the DBA to add, edit, and delete schema definitions. The schema definition is fundamental to the relational data dictionary. The schema can be considered a logical group of tables, related by owner or function. Great care should be taken before modifying any schema definition.

If you have not defined any schemas, the Add Schema window appears. Select YES to add a schema. Otherwise, if schemas exist, the Select, Insert, Delete selection window appears. You can press [insert] to add a new schema or select the schema you wish to edit. If you want to delete a schema, highlight it and press [delete].

ADD

Schema definitions should be added in the beginning of the global mapping process. Once a schema definition has been defined, you may create tables within that schema.

EDIT

Schema definitions can be edited with caution. Any references to the former schema name are flagged as errors. If the global name is changed, be sure to verify the status of any data stored under the former name. There is no automatic transfer of data to the new global name.

DELETE

You may not delete a schema that is referenced by tables in your system. The delete function should be used with caution in those circumstances where a schema was entered by mistake.

Schema Information

```
Schema Information
Schema name: ACCOUNTING_____________
Description: Accounting data dictionary_____________________________
Global name: SQLACCT_____________________
Filer base routine: _________
```

Schema name : **character (30)**

The name is the logical reference for this schema definition. Each schema must have a unique name.

Description : **character (60)**

The schema description appears on the list of schema definitions.

Global name : **character (10)**

If users are allowed to create tables using the CREATE command, you can specify a global to store the table data for this schema. If this value is not defined, the values are stored in the default global for the site

Filer base routine : **character (5) [list]**

These are base routines used for table filers and are similar to the base routines used for queries. We recommend you use base routines for separation of object types. The data characteristics and length are inherited from BASE ROUTINE EDIT. If you do not specify a filer base routine, the system uses the default base routine for the site.

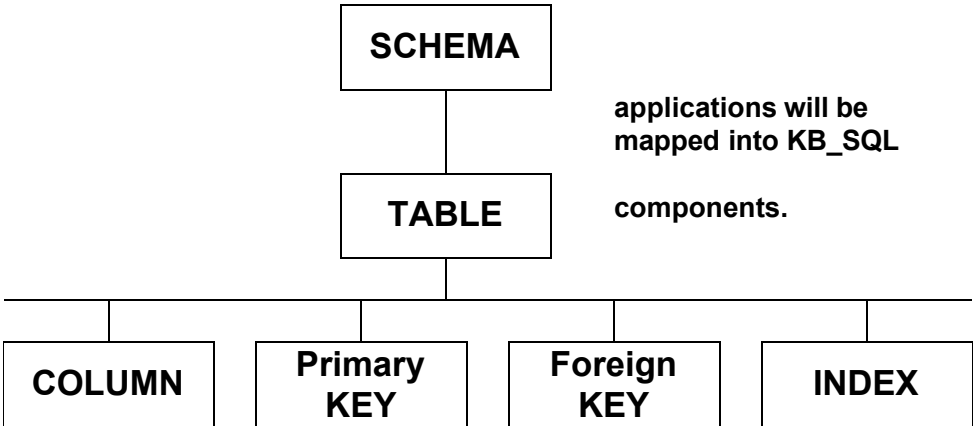
MAP EXISTING GLOBALS option

```
Select DATA DICTIONARY
MAP EXISTING GLOBALS
DOMAIN EDIT
KEY FORMAT EDIT
OUTPUT FORMAT EDIT
SCHEMA EDIT
REPORTS
```

Note: Our preferred tool for mapping globals is the DDL interface (explained in Chapter 5) which lets you translate a foreign M data dictionary into a KB_SQL data dictionary using a script file. This section explains KB_SQL's initial method for mapping M globals.

In order to refer to your data using SQL, you must represent your M globals as a relational data dictionary. Having done any necessary preliminary work of defining schemas, domains, output formats, and key formats, you may begin to define the tables and columns of your database.

The KB_SQL Data Dictionary



applications will be mapped into KB_SQL components.

Suggested Procedure for Mapping Globals

When mapping globals we suggest the following procedure:

1. Use the COLUMNS option to create the columns you want to be the primary key columns.
2. Select the PRIMARY KEYS option and let KB_SQL automatically build the primary key definition(s).
3. Use KB_EZQ to generate a query that references the primary key(s). Check that you are getting the correct primary key data.
4. Define the remaining columns using the COLUMNS option. Define a few columns at a time, using KB_EZQ to check them before defining additional columns.

Adding/Editing/Deleting Tables

A table is a set of related rows where each row has a value of one or more columns of the table. Rows are distinguished from one another by having a unique primary key value. To add, edit, or delete tables, select MAP EXISTING GLOBALS from the Select DATA DICTIONARY window.

ADD

To add a table, enter a name at the Table prompt in the Schema and Table Name window. After you have supplied all the information pertaining to the table (creating columns and specifying the primary key columns), the Commit? prompt appears. If you want to save all the information you entered, type Y and press [enter]. Users can then write queries and retrieve information from the table you added.

EDIT

You can edit a table at any time. Existing queries must be recompiled if you delete columns, change primary keys, or otherwise alter the current definition of the table.

DELETE

You can delete a table but queries that reference the table will have to be altered to reference another table, and then the queries must be recompiled. To delete a table, highlight the table's name from the selection window and press [delete].

Schema and Table Name

If the table exists, you may skip the schema prompt and enter the table name directly. If you enter a name of a table that does not exist, the Add TABLE window appears after you specify a schema. Select YES to add the new table.

```
Schema and Table Name
Schema: SQL_TEST____________________
 Table: ____________________________
Select, Insert, Delete Table
*CHARGES
 EMPLOYEES
 INCOMPLETE_TASKS
 PROJECTS
 TASKS
```

Schema : **character (30) [list]**

Use the [list] function or enter a partial match to a schema name. Selecting a schema restricts the scope of the table list to include only those tables in the selected schema.

Table: **character (30) [list]**

Use the [list] function or enter a partial match to a table name. A schema name must be specified in order to add a new table.

Table Options

The MAP EXISTING GLOBALS procedure is organized with an internal menu system to streamline the input process. The COLUMNS option is highlighted as the default option. You may use either [skip] or the EXIT option to exit the mapping procedure.

```
Table Options
TABLE INFORMATION
*COLUMNS
PRIMARY KEYS
FOREIGN KEYS
INDICES
EXIT
```

TABLE INFORMATION Option

The TABLE INFORMATION option allows you to edit the name, schema, density, and description of the table.

Table Information

```
Table Information
Table: CHARGES_____________________ Schema: SQL_TEST_____________________
```

Table: **character (30)**

The table name is the logical reference for the table definition. The name must be a valid SQL_IDENTIFIER.

Schema: **character (30) [list]**

You can change the schema that the table is associated with by selecting a different schema name at this prompt. All table and index information is associated with the new schema.

Table Description

```
Table Information
Table: CHARGES_____________________ Schema: SQL_TEST_____________________

Table Description
This is a table of time charged by employees to projects.
```

Table Description: **character (60)**

The table description appears on the TABLE PRINT report (DATA DICTIONARY/REPORTS/TABLE PRINT).

Table Features

```
Table Features
 Primary key delimiter: ___
     Default delimiter: ___
               Density: _________________
         Allow updates? NO_
Last assigned sequence= ____
```

Primary Key Delimiter: **character (3)**

This value is optional and is only used for tables with more than one primary key column. In these cases, a single composite string is created from all the primary key columns. The delimiter is a character that can be used to separate the primary key components in that composite string. The character must not be contained in any primary key value. The tab character is the default character for this purpose; however, you must take care that it also is not contained in any primary key value.

Default Delimiter: **integer (3)**

The table default delimiter is the ASCII value for the default character used to separate data values. For example, the default delimiter for a semicolon is 59. You can assign a default delimiter for each site (refer to the discussion on the SITE EDIT option in the *KB_SQL Database Administrator's Guide*)

and for each table within each site. If you specify a table default delimiter, it overrides the site default delimiter. If you do not specify a table default delimiter, the system refers to the site default delimiter.

Density: **character (20) [list]**

To obtain the cost of a query, the optimizer takes the cost of traversing a table and divides it by the table's density value. We recommend that instead of using this prompt to assign a density value for a particular table, you assign density values at the site level (using the CONFIGUARTION/SITE EDIT/ACCESS PLAN INFO option) which will apply to all tables.

Last Assigned Sequence= **integer (4)**

This value represents the highest sequence number that has been assigned to a column in this table. If you want a column to be updatable, you assign it a sequence number which is referenced by the table filer program. You can assign a column's sequence number by using the Real Column Information window. Refer to the section on Writing the Table Filer in Chapter 4:Table Filers to see how column sequence numbers are used by the table filer program to update a table.

Allow Updates? **YES/NO**

Answer YES if you intend to allow updates to the data in this table via the INSERT, UPDATE, and DELETE commands.

IMPORTANT:

You must write a table filer routine in order to execute updates. Refer to Chapter 4 in this guide for more information on the subject.

The Update Table Logic window appears if you answer YES to the *Allow updates* prompt.

<u>Update Table Logic</u>

```
Update Table Logic
INSERT FILER
UPDATE FILER
DELETE FILER
READ LOCK
READ UNLOCK
WRITE LOCK
WRITE UNLOCK
EXIT
```

For each of the options on the menu, a drop-down window appears in which you supply the appropriate M execute code to perform the corresponding action. Refer to Chapter 4 for more information.

The Read Lock Logic window appears if you answer NO to the *Allow updates* prompt. It lets you specify M execute code to read lock a row and read unlock a row.

<u>Read Lock Logic</u>

```
Read Lock Logic
 READ LOCK
 READ UNLOCK
*EXIT
```

Compiling Table Statistics

In an effort to ensure that statistics are run for each table, the Compile Table Statistics window appears when you exit from the Table Options menu. The KB_SQL query access planner uses a table's statistics to determine the best path for a query. For more information on calculating statistics or to calculate statistics for all the tables in a schema, refer to the *KB_SQL Database Administrator's Guide.*

Compile Table Statistics

```
Compile table statistics?
*   If needed        Now        No (incomplete)
```

If needed

Compile only if statistics do not exist and table is referenced by a query.

Now

Compile statistics for this table now (add process to background queue).

No

Don't compile statistics, table cannot be used by queries.

COLUMNS Option

```
Table Options
 TABLE INFORMATION
*COLUMNS
 PRIMARY KEYS
 FOREIGN KEYS
 INDICES
 EXIT
```

Use the COLUMNS option to add, edit, or delete the basic information about the columns in the table.

ADD

Add a column by selecting the COLUMNS option, assigning a name to the column, and supplying the necessary information in the windows that follow. After you add that column, it can be referenced by queries.

EDIT

If you change a column definition, any query that referenced the table that contains the column must be recompiled.

DELETE

If you delete a column definition, any query that referenced the table that contains the column must be recompiled. Also any reference in the query to the deleted column must be changed to reference an existing column. To delete a column, highlight the column's name in the selection window and press [delete].

Column Name

```
Table Information
Table: CHARGES_____________________ Schema: SQL_TEST_____________________
```

```
Column Name
Column name: TASK_NO_____________________
```

Column name : **character (30) [list]**

The name must be a valid SQL_IDENTIFIER. If you wish to edit or delete a column name, use the [list] function or enter a partial match to the column name. The list includes all matching columns from the table except foreign key columns.

Column Description

```
Column Description
This is the task identifier.
```

Column description: **character (60)**

The column description appears in column selection windows and on the TABLE PRINT report.

Column Information

Confidential | ©2023

```
Column Information
              Domain: INTEGER____________________
              Length: 2__ Scale: _
       Output format: ___________________________
            Required? YES
      Default header: ___________________________
For programmers only? NO_      Conceal on SELECT *? NO_
     Change sequence: ____  Last assigned sequence= ____
      Virtual column? NO_
```

Domain : **character (30) [list]**

Each column definition must be linked to a domain. The domain specifies the internal (global) storage format of the data.

Length : **integer (3)**

The default length of a column definition is the length of the associated domain or data type. A different length may be specified.

Scale: **integer (1)**

For numeric data types, you must also enter the number of digits to the right of the decimal point.

Output format: **character (30) [list]**

The name must be a valid SQL_IDENTIFIER. You can press [list] to view the list of output formats. Each data type has its own set of output formats. A character data type has three formats to choose from: INTERNAL, LOWER, or UPPER. INTERNAL is free text, no limitations; LOWER refers to lowercase; and UPPER refers to uppercase.

Required: **YES/NO**

Answer YES, if every row must have a non-null value for this column. Answer YES, if this column is a primary key column for this table.

Default header: **character (30)**

By default, KB_SQL displays the column name as the heading over a column of data in a query result. You can override this default by specifying a default header text. You can include the vertical bar (|) to force a multi-line heading.

For programmers only? **NO/YES**

Certain columns are defined for use by programmers only. Answer YES, to restrict the view of this column to programmers. If you answer YES, the column is accessible only through the SQL Editor. For example, you may want to restrict access to DATA_1 which is the string of all columns stored on the first node of a global.

Conceal on SELECT * ? **NO/YES**

For tables with ten or more columns, the display produced by a

SELECT * command can be difficult to interpret. Answer YES to conceal extraneous or programmer columns from the display.

Change sequence : **integer (4)**

This value is a unique identification number that you assign to the column. The SQL INSERT and UPDATE statements use this number to indicate which column values have been changed. Therefore, a value must be assigned if you wish to allow updates to this column. If you do not wish to allow updates to the column, set this value to zero. If you do not enter a value, and you use the TABLE^SQL0S utility to generate a filer routine, the utility will assign the next available sequence number.

Note: *If a table is not modifiable, this prompt has no effect.*

Last assigned sequence= **integer (4)**

This value represents the highest change sequence number that has been assigned to a column in this table. If you want a column to be updatable, you assign it a change sequence number which is referenced by the table filer program. Refer to the section on Writing the Table Filer in Chapter 4:Table Filers to see how column sequence numbers are used.

Virtual column? **NO/YES**

Since most columns have a physical storage requirement, the default answer is NO. A NO answer will result in the appearance of the Real Column Information window. Answer YES, if this column is a virtual column having no physical storage requirement in this table. A YES answer will bring up the Virtual Column Definition window.

Virtual Column Definition

```
Column Name
Column name: MANAGER_NAME_______________

Virtual Column Definition (SQL Expression)
MANAGER_LINK@NAME
```

Virtual column definition: **character (180)**

Enter a valid SQL expression for a virtual column. The expression can reference other columns and functions using valid SQL syntax. This example shows how to use a foreign key reference as a virtual column.

Real Column Information

```
Column Name
Column name: MANAGER_____

Real Column Information
   Parent column: DATA_____
 Global reference: _____
  Piece reference: ";",3)_____
     Extract from: ___   To: ___
```

Parent column: **character (30) [list]**

Select a parent column, if the definition of this column is dependent on a previously defined column. A parent column is often specified for the primary keys 2-n and for data columns (those having a horizontal suffix).

Global reference: **character (55)**

The global reference is the string of characters in the M global address that come after any previously defined columns and before the current column. For example, the first primary key often has the global name as the vertical prefix. Each successive key specifies only the comma (,) that separates the M subscripts.

Piece reference: **character (55)**

The piece reference is the string of characters in the M global address that completes the $PIECE function reference for this column. For example, the string ";",2) specifies the second semicolon piece.

Note: *If you had specified a default delimiter for this table or this site, you would need to type only the number of the piece (e.g., 2). Refer to the Table Features window or the SITE EDIT option for more information on default delimiters.*

Extract from :____To: __

If your data values are not delimited, you can use the extract prompt to specify a starting (from) and ending (to) point for this data value. This is an alternative to using the EXTRACT() function in a virtual column definition, and it provides for more optimal code generation**.**

IMPORTANT*:* You cannot use both the piece reference and the extract.

PRIMARY KEYS Option

```
Table Options
 TABLE INFORMATION
 COLUMNS
*PRIMARY KEYS
 FOREIGN KEYS
 INDICES
 EXIT
```


Confidential | ©2023

The PRIMARY KEYS option allows you to specify those columns that are required to reference a row from the table. If you have not previously created any primary keys for a table, KB_SQL attempts to automatically create them based on the columns you have created. The message, “ Default primary key created,” appears at the bottom of the screen. If KB_SQL can not automatically create them, the message

“ Unable to create default primary keys” appears.

If KB_SQL was unable to create default primary keys and you have not defined any primary keys for this table/schema, the Add Primary Key window appears. Select YES to add a primary key. Otherwise, if primary keys do exist, the Select, Insert, Delete selection window appears. KB_SQL displays the primary keys in sequence number order. You can press [insert] to add a new primary key or select the key you wish to edit. If you want to delete a key, highlight it and press [delete].

To build primary keys for a table, the primary key columns are required to have a vertical prefix and must not have a horizontal suffix. They must have a global reference and must not have a piece or extract reference. Custom primary keys do not have a vertical or a horizontal address. The definition comes from the primary key custom logic.

Primary Key Column

Each primary key must be assigned a sequence number and be linked to a column from the table. The primary key with sequence number 1 is considered to be the most significant key.

```
Table Information
Table: CHARGES_____________________ Schema: SQL_TEST_________________

Primary Key Column
Key sequence: 3_    Key column: CHG_DATE_________________
```

Key sequence: **integer (1)**

The sequence number for a primary key is a way of specifying the significance of the key. The sequence numbers range from 1 to 9, indicating highest to lowest significance, ordered from left to right.

Key column: **character (30) [list]**

Select a column from a list of columns from the active table. The list includes all real columns.

Primary Key Information

Confidential | ©2023

```
Primary Key Column
Key sequence: 3_     Key column: CHG_DATE_____________________

Primary Key Information
          Start at value: ___________________
            End at value: ___________________
          Avg subscripts: 5.20_____
Skip search optimization? NO_
       Allow NULL values? NO_
```

Start at value: **character (20)**

Specify a starting string, if the generated M code for traversal of this primary key should start at a value other than NULL. This value is assigned into a looping variable before the first row retrieval. Be sure to enclose character literal values in double quotes.

End at value: **character (20)**

Specify an ending string, if the generated M code for traversal of this primary key should end at a value other than NULL. Be sure to enclose character literal values in double quotes.

Avg subscripts: **character (20) [list]**

You can specify the average number of unique values for this primary key. This number is used in a calculation that determines the overall

cost of traversing a table. Valid input for this prompt includes numeric values or predefined character string (fuzzy) values (e.g., small, medium, large). You can define the fuzzy values by using the UTILITIES/STATISTICS/FUZZY SITE EDIT option. If left blank, the system uses the default average number of distinct entries that is specified for your site.

In the example shown, there is an average of 5.20 charge dates per primary key #3.

Note: *Unless you completely understand optimization, we recommend you run the statistics compiler (UTILITIES/STATISTICS/COMPILE STATISTICS) and have the system supply a value for Avg subscripts.*

Skip search optimization? **NO/YES**

If the primary key cannot be optimized, then enter YES; otherwise, enter NO. Two reasons for not optimizing a primary key are: 1) if the key collates as a string instead of as a numeric, and 2) if you have provided custom primary key logic that alters the standard logic to an extent that optimization of the key would produce erroneous results. If you have altered the primary key logic and are not sure whether or not you should skip optimization, consult your KB_SQL technical support representative.

As an example, consider the MOMENT domain which is a date and time stamp. Because of the way we treat this time stamp as $H, it collates as a string, and therefore cannot be optimized for searching. The MOMENT domain is set to skip search optimization. (The *Skip search optimization* prompt can be set at the domain level in the Domain Information window. Refer to the DOMAIN EDIT option.)

You could create your own domain based on MOMENT that does collate correctly. For example, if your MOMENTS collate like numbers, search optimization applies. You can override the skip setting in your domain to skip search optimization for a primary key by setting this *Skip search optimization* prompt to YES.

Note: *Optimization applies only to the equal to, greater than, less than, or in operators. If KB_SQL tries to optimize a key that shouldn't be optimized, the query returns the wrong results.*

Allow NULL values? **NO/YES**

If your system allows null values as subscripts and if this key may be null, enter YES; otherwise, enter NO. For more information on null values, refer to Appendix B in the *KB_SQL Database Administrator's Guide*.

Primary Key Logic

The majority of M globals will not require any primary key logic. Usually, the primary key columns are the subscripts of a global. Row traversal is accomplished by the $ORDER function. Sometimes, M globals include repeating groups of data in a field separated by delimiters. Since the relational model does not allow a column to contain repeating groups, the group must be defined as a table. Each member of the group is treated as a separate row. The primary key logic provides a way for you to customize the default logic for row traversal.

```
Primary Key Logic
PRE-SELECT
CALCULATE KEY
VALIDATE KEY
END CONDITION
POST-SELECT
INSERT KEY COMPUTE
*EXIT
```

The primary key logic consists of M executables and expressions that allow you to write your own row traversal logic. The following parameters can be referenced by the primary key logic.

Compile-Time	Description
{KEY(#)}	Key value for other primary key for this table
{KEY}	Current primary key value
{VAR(#)}	Temporary variable for this table

Primary Key Logic (standard)

The following frame shows an example of standard primary key logic. Note how the looping starts and ends with the empty string or NULL value.

```
PGM         ; sample primary key logic (standard)
            ;
            ; pre-select
            S K1=""
            ; calculate key
A           S K1=$O(^TEST(K1))
            ; end condition
            I K1="" G B
            ; logic
            ;
            G A
B           ;
```

Primary Key Logic (Custom)

The following frame contains an example of custom primary key logic. Note the placement of the custom logic executes. You may traverse complex primary keys using these executes and conditions.

```
PGM         ; sample primary key traversal (custom)
            ;
            ; pre-select execute
            S X1=^TEST(K1),X2=$L(X1,";"),X3=0
A           ; calculate key
            S X3=X3+1,K2=$P(X1,";",X3)
            ; end condition
            I X3>X2 G B
            ; validate key
            I K2="*" G A
            ; post-select execute
            I K2["*" S K2=$TR(K2,"*")
            ; logic
            ;
            G A
B           ;
```

PRE-SELECT Option

The code shown in the Pre-select Execute window below is executed prior to the beginning of the traversal loop. As an example, consider a programmer column of codes separated by semicolons, where each code is a primary key value. The pre-select execute code could save the number of codes in a scratch variable for reference in the end condition.

Confidential | ©2023

```
Table Information
Table: TEST_TABLE_____________________ Schema: SQL_TEST_____________________

Primary Key Column
Key sequence: 2_     Key column: KEY2_____________________

Pre-select Execute (M Execute)
S {VAR(1)}=^TEST({KEY(1)}),{VAR(2)}=$L({VAR(1)},";"),{VAR(3)}=0
```

CALCULATE KEY Option

The default method for traversal of primary keys is the M $ORDER function. For complex primary keys, you may specify custom logic for getting the next key. In our example, we increment a counter for stepping through the pieces of a data column.

```
Table Information
Table: TEST_TABLE_____________________ Schema: SQL_TEST_____________________

Primary Key Column
Key sequence: 2_     Key column: KEY2_____________________

Calculate KEY Value (M Execute)
S {VAR(3)}={VAR(3)}+1,{KEY}=$P({VAR(1)},";",{VAR(3)})
```

VALIDATE KEY Option

This execute can be used to determine if a selected key value is a valid primary key value. Continuing with our example, consider if certain code pieces are equal to the asterisk (*) character. Enter an M condition on which the key would be skipped if the test fails.

```
Table Information
Table: TEST_TABLE_____________________ Schema: SQL_TEST_____________________

Primary Key Column
Key sequence: 2_     Key column: KEY2_____________________

Valid KEY Condition (M Expression)
{KEY}'="*"
```

END CONDITION Option

The primary key traversal logic terminates on either the value provided in the *End at value* prompt in the Primary Key Information window or the empty string. You may specify additional criteria to terminate the loop. For example, you may specify the loop should be terminated if the key counter exceeds the number of keys in the string.

```
Table Information
Table: TEST_TABLE_____________________ Schema: SQL_TEST_____________________

Primary Key Column
Key sequence: 2_     Key column: KEY2_____________________

End of KEY Condition (M Expression)
{VAR(3)}>{VAR(2)}
```

POST-SELECT Option

In some cases, even a valid key may need some manipulation. This execute does not affect the looping logic since the code is applied after the key has been selected. This example strips all occurrences of the asterisk (*) character from the key value.

```
Table Information
Table: TEST_TABLE_____________________ Schema: SQL_TEST_____________________

Primary Key Column
Key sequence: 2_     Key column: KEY2_____________________

Post-select Execute (M Execute)
I {KEY}["*" S {KEY}=$TR({KEY},"*")
```

INSERT KEY COMPUTE Option

This option is related to the use of table filers and comes into play when you insert a new row into the table. At that point, you want to associate a unique key for the new row. The NEXTID function shown in the Compute Key on Insert window below generates that unique key.

```
Table Information
Table: TEST_TABLE_____________________ Schema: SQL_TEST_____________________

Primary Key Column
Key sequence: 2_     Key column: KEY2_____________________

Compute KEY on Insert
S {KEY}=$$NEXTID()
```


Confidential | ©2023

FOREIGN KEYS Option

```
Table Options
TABLE INFORMATION
COLUMNS
PRIMARY KEYS
*FOREIGN KEYS
INDICES
EXIT
```

KB_SQL extends the ANSI definition of foreign keys to provide an efficient method for joining tables by primary key values. Each foreign key definition includes a named set of columns that match the primary key of another table. Foreign keys can point to any row, including another row in the same table or even to the same row (as the primary key) in the same table.

If you have not defined any foreign keys for this table/schema, the Add Foreign Key window appears. Select YES to add a foreign key.

Otherwise, if foreign keys do exist, the Select, Insert, Delete selection window appears. You can press [insert] to add a new foreign key or select the key you wish to edit. If you want to delete a key, highlight it and press [delete].

Foreign Key

```
Table Information
Table: CHARGES                    Schema: SQL_TEST

Foreign Key
  Foreign key name: PROJECT_LINK
Reference to table: PROJECTS
```

Foreign key name : **character (30)**

The name is the logical reference for this foreign key column. You may wish to use a naming convention for foreign key names, such as ending all names with '_LINK'.

Reference to table : **character (30) [list]**

By definition, a foreign key is equal to some other primary key. Select the table to which this foreign key applies. You are asked to specify a foreign key column to match each primary key of the referenced table.

Foreign Key Description

```
Foreign Key Description
This is a link to the PROJECTS table.
```

Foreign key description: **character (60)**

The foreign key description appears on the TABLE PRINT report and in column selection windows.

Foreign Key Column

```
Table Information
Table: TASKS                    Schema: SQL_TEST

Foreign Key
  Foreign key name: PROJECT_LINK
Reference to table: PROJECTS
Foreign Key Column
Key sequence= 1_   Primary key column= PROJ_NO
                   Foreign key column: PROJ_NO
```

Key sequence and Primary key column

The foreign key sequence and associated primary key is displayed. The second foreign key column matches the second primary key column from the referenced table.

Foreign key column: **character (30) [list]**

Select the column from the active table that equates to the displayed primary key column from the referenced table.

INDICES Option

```
Table Options
TABLE INFORMATION
COLUMNS
PRIMARY KEYS
FOREIGN KEYS
INDICES
EXIT
```

Indices are a special type of table that can be used by KB_SQL to optimize query execution. You can define one or more indices for a base table. The index provides another way to determine the base table's primary key values.

- An index is an M global.
- An index cannot include data that is not in the base table.
- An index must include all of the primary keys of the base table as columns.
- Once defined, an index can be used implicitly by the access planner to satisfy requests for information from the base table.
- Once defined, an index can be used explicitly in a FROM clause (or as the value for the *Use table* prompt in EZQ) to satisfy requests for information from the index table.

We recommend that users write their queries accessing the base table and trust the KB_SQL optimizer to select the correct index table. To eliminate any possible confusion, the DBA could give users access to the base table but not access to any corresponding indices.

An Example of Using an Index

The table:

PART (base table)

columns (p_no, name, cost)

global ^P(p_no) = name ^ cost

The index:

PART_BY_NAME (index table)

columns (name, p_no)

global ^P(“ A” ,name,p_no) = “”

The query:

SELECT p_no, name, cost

FROM part

WHERE name BETWEEN ‘A’ and ‘KZZ’

The plan:

Get table PART

Using Index PART_BY_NAME

Optimize primary key (name)

The result:

The access planner chooses to use the PART_by_NAME index to satisfy the query.

If you have not defined any indices for a table/schema, the Add Index window appears. Select YES to add an index. Otherwise, if indices do exist, the Select, Insert, Delete selection window appears. You can press [insert] to add a new index, or select the index you wish to edit. If you want to delete an index, highlight it and press [delete].

Confidential | ©2023

Table Information

```
Table Information
Table: EMPLOYEES_____________________ Schema: SQL_TEST_____________________

Select, Insert, Delete Index
*EMP_BY_NAME
```

If you are adding a new index, the Table Index window appears; otherwise, if you are editing an index, the Index Options window appears.

Index Options

Since KB_SQL allows indices to be used like any other table, the definition of an index is very similar to that of a base table. By defining an index for a table, you provide more information to be used by the data access planner during compilation of queries.

```
Table Index
Index: EMP_BY_NAME_________________ Table: EMPLOYEES_____________________

Index Options
 INDEX INFORMATION
*INDEX COLUMN
 INDEX PRIMARY KEYS
 INDEX FOREIGN KEYS
 EXIT
```

INDEX INFORMATION Option

Table Index

The INDEX INFORMATION option allows you to modify the name, density, and description of the index.

```
Table Index
Index: EMP_BY_NAME_________________ Table: EMPLOYEES_____________________
```

Index: **character (30)**

The index name is the logical reference for the index definition. The name must be a valid SQL_IDENTIFIER.

Table: **character (30) [list]**

Enter the table name to which this index applies. The name must be a valid SQL_IDENTIFIER.

Index Description

```
Table Index
Index: EMP_BY_NAME_________________  Table: EMPLOYEES_________________

Index Description
employees by name index
```

Index description: **character (60)**

Enter a description for this index. The description appears on selection windows and in the TABLE PRINT report.

Index Information

```
Table Index
Index: EMP_BY_NAME_________________  Table: EMPLOYEES_________________

Index Information
Skip index optimization? NO_
  Number of unique keys: _
          Index density: _________
```

Skip index optimization? **YES/NO**

Enter YES if this index is defined only in certain situations. If you enter YES, this index is not considered as a candidate in the query access planning process.

Number of unique keys: **integer (1)**

See Chapter 4: Table Filers for information on this prompt.

Index density: **character (20) [list]**

Select from a list of fuzzy density values or enter a numeric value for the relative density of this index. The density is roughly equated to the number of entries (rows) per physical M data block. We recommend that instead of using this prompt to assign a density value for a particular index, you assign density values at the site level (using the CONFIGUARTION/SITE EDIT/ACCESS PLAN INFO option) which will apply to all indexes.

INDEX COLUMN Option

If you have not defined any index columns, the Add Index Column window appears. Select YES to add an index column. Otherwise, if index columns exist, the Select, Insert, Delete selection window appears. You can press [insert] to add a new index column or select the index column you wish to edit. If you want to delete an index column, highlight it and press [delete].

Index Column

```
Table Index
Index: EMP_BY_NAME_________________  Table: EMPLOYEES_____________________

Index Column
        Column name: NAME_______________________
Conceal on SELECT *? NO_
      Parent column: _________________________
   Global reference: ^SQLEMPN(_____________________________
    Piece reference: ________________________________________
       Extract from: ___  To: ___
```

Conceal on SELECT * ? **YES/NO**

For tables with ten or more columns, the display produced by a

SELECT * command can be difficult to read. Answer YES, to conceal extraneous or programmer columns from the display.

Parent column: **character (30) [list]**

Select a parent column, if the definition of this column is dependent on a previously defined column. A parent column is often specified for the primary keys 2-*n* and for all data columns.

Global reference: **character (20)**

The global reference is the string of characters in the M global address that come after any previously defined columns and before the current column. For example, the first primary key often has the global prefix as the vertical prefix. Each successive key specifies only the comma (,) that separates the M subscripts.

Piece reference: **character (20)**

The piece reference is the string of characters in the M global address that complete the $PIECE function reference for this column. For example, the string ";",2) specifies the second semi-colon piece.

Extract from : ___To: __

If your data values are not delimited, you can use the extract option to specify a starting (from) and ending (to) point for this value. This is an alternative to using the EXTRACT() function in a virtual column definition, and it provides for more optimal code generation.

IMPORTANT : You cannot use both the piece reference and the extract.

INDEX PRIMARY KEYS Option

If KB_SQL was unable to create default primary keys for this index, and if you have not defined any primary keys for the index, the Add Primary Key Column window appears. Select YES to add a primary key. Otherwise, if index primary keys exist, the Select, Insert, Delete selection window appears. You can press [insert] to add a new primary key, or select the primary key you wish to edit. If you want to delete a primary key, highlight it and press [delete].

Index Primary Key Column

```
Table Index
Index: EMP_BY_NAME_________________  Table: EMPLOYEES_________________

Index Primary Key Column
Key sequence: 1_    Key column: NAME_____________________
```

Key sequence: **integer (1)**

The key sequence is the order of significance from 1-9, indicating the order from left to right in the global subscripts.

Key column: **character (30) [list]**

Each index column must be linked to a source column from the base table. Each occurrence of the source column in the base table is paired with a row in the index table.

Index Primary Key Information

```
Index Primary Key Information
            Key Format: ___________________________
           Sort NULL as: ___________________________
         Start at value: ________________
           End at value: ________________
         Avg subscripts: 8________
Skip search optimization: NO_
      Allow NULL values: NO_
```

Key format: **character (30) [list]**

By default, the index column value is stored in the same format as in the base table. If the stored format in the index is different, you may select from a list of defined key formats.

Sort NULL as: **character (20)**

By default, the system expects that NULL values in base tables are not represented in the corresponding index tables. If the index does include NULL values, enter the string of characters that is used. Remember to include double quotes around character literal values.

Start at value: **character (20)**

Specify a starting string if the generated M code for traversal of this primary key should start at a value other than NULL. This value is

assigned into a looping variable before the first row retrieval. Be sure to enclose character literal values in double quotes.

End at value: **character (20)**

Specify an ending string if the generated M code for traversal of this primary key should end at a value other than NULL. Be sure to enclose character literal values in double quotes.

Avg subscripts: **character (20) [list]**

You may specify the average number of unique values for this primary key using numbers or fuzzy size values. If left blank, the system uses the default average number of distinct entries as specified for your site.

Skip search optimization: **NO/YES**

If the primary key cannot be optimized by applying the greater than, less than, or contains operators, then enter YES; otherwise, enter NO.

Allow NULL values: **NO/YES**

If your system allows null values as subscripts and if this key may be null, enter YES; otherwise, enter NO.

Note: *For more information on null values, refer to Appendix B in the KB_SQL Database Administrator's Guide.*

Index Primary Key Logic

Primary key logic for index tables is similar to the logic for base tables. Refer to the discussion on Primary Key Logic earlier in this chapter.

Primary Key Logic

```
Primary Key Logic
 PRE-SELECT
 CALCULATE KEY
 VALIDATE KEY
 END CONDITION
 POST-SELECT
*EXIT
```

INDEX FOREIGN KEYS Option

Generally, an index contains a foreign key that points back to the base table. This allows the index to be viewed as logically equivalent to the base table.

If you have not defined any foreign keys for the index, the Add Foreign Key window appears. Select YES to add a foreign key. Otherwise, if index foreign keys exist, the Select, Insert, Delete selection window appears. You can press [insert] to add a new foreign key or select the foreign key you wish to edit. If you want to delete a foreign key, highlight it and press [delete].

Table Index

```
Table Index
Index: EMP_BY_NAME                    Table: EMPLOYEES
```

```
Select, Insert, Delete Foreign Key
*EMP_LINK
```

The windows that you see during this process are similar to those used to add/edit a foreign key. Refer to the section “FOREIGN KEYS Option” for more detailed instructions.

REPORTS Option

```
Select DATA DICTIONARY
MAP EXISTING GLOBALS
DOMAIN EDIT
KEY FORMAT EDIT
OUTPUT FORMAT EDIT
SCHEMA EDIT
REPORTS
```

The data dictionary reports are valuable tools to be used by the DBA and others during the global mapping process. Each report can be used to validate the structures defined at certain checkpoints in the process. The reports include any expressions or executes that are used for data transformations in your system. A hard copy of each report should be available to the DBA at all times.

Select REPORTS

```
Select REPORTS
DOMAIN PRINT
KEY FORMAT PRINT
OUTPUT FORMAT PRINT
SCHEMA PRINT
TABLE PRINT
VIEW PRINT
```

DOMAIN PRINT Option

This procedure can be used to print a report of all data storage methods defined for your system. The report includes all domain parameters and all transform expressions or executes for a range of domain names.

KEY FORMAT PRINT Option

This procedure can be used to print a report of all key formats defined for your system. The report includes all key format parameters and all transform expressions or executes for a range of key formats.

OUTPUT FORMAT PRINT Option

This procedure can be used to print a report of all output formats defined for your system. The report includes the name, length, and justification parameters for a range of output format names.

SCHEMA PRINT Option

This procedure can be used to print a list of all application and user group schemas defined for your system.

TABLE PRINT Option

This procedure can be used to print the logical table definitions for a selected schema. The report includes the definitions for all columns, primary keys, foreign keys, and indices for a range of table names. If desired, the report prints the physical definitions, including vertical and horizontal addresses.

This report is a valuable tool during the process of mapping to existing globals. If the report does not show the table and column definitions the way you expect to see them, you should review the definitions. You can avoid time-consuming debugging by ensuring that the table printout is correct as a checkpoint in the mapping process.

This report can also be used to view the table and column definitions for the KB_SQL Data Dictionary (the DATA_DICTIONARY schema).

Print Tables

```
Print Tables
    Schema name: ____________________________
From table name: A___________________________
Thru table name: ZZZ_________________________
  Print globals? YES
 Break at table? NO_
```

Schema: **character (30) [list]**

Use the [list] function or enter a partial match to a schema name. Selecting a schema restricts the scope of the table list to include only those tables in the selected schema.

From table name: **character (30) [list]**

Thru table name: **character (30) [list]**

You can define a range of tables to be included in the report. You may either enter beginning and ending values, or you can press [list] at each prompt to select from the list of tables.

Print globals? **YES/NO**

Answer YES if you want the report to include the physical definitions.

Break at table? **YES/NO**

Answer YES if you want a page break to occur at the end of each table's definition.

VIEW PRINT Option

This procedure can be used to print a list of all view names and the SQL text needed to build each view.

Chapter 4: Table Filers

KB_SQL Version 3 provides *full* database management using SQL. The management of legacy globals is addressed through a technology called *table filers* — M routines that apply changes to a database.

Much of the terminology used in this chapter is documented and demonstrated in the *KB_SQL Programmer's Reference Guide*. We urge you to review that manual before attempting to use table filers.

Overview

KB_SQL separates SQL statement and column validation from the table row validation and filing operations. The M routine generated by the INSERT, UPDATE, or DELETE statement creates a value array global and performs row locking and column level validation, including required columns and data type checks. A table filer routine performs row level validation, including referential integrity and unique indices, and updates the M globals with the changes specified in the value array.

Table filer routines generate automatically for tables produced using the CREATE TABLE statement.

All other tables require a hand-coded table filer routine.

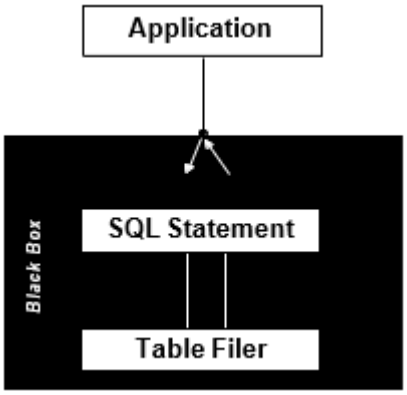
Hand-coded table filers must conform to this specification document. Custom execute logic must be added to table definitions using the MAP GLOBALS option, and to domains using DOMAIN EDIT.

These topics are examined more fully later. Before we begin, let's establish the basis for that discussion.

Preliminaries

In this chapter we refer to your code as the application. Any SQL code you reference using ESQL or API/M is referred to as an SQL statement. The SQL statement automatically calls table filers as necessary.

From the application's perspective, the work performed by the SQL statement is indistinguishable from the work performed by the table filer. In other words, your application's code treats the SQL statement and table filer as a single black box, shown right.



Four sections comprise this chapter: a brief description of commonly used terms; a discussion of the relevant SQL statements; a section that describes the table filers that are automatically built for tables created by DDL statements; and finally, a section that reviews the steps necessary to write a table filer for a table defined by the MAP EXISTING GLOBALS option.

***Note**: Once the relevant information exists, you may produce printouts of table definition reports and table filer routines similar to those shown in this chapter using the DATA DICTIONARY/ REPORTS/TABLE PRINT option.*

Terminology

Term	Definition
Business rules	A type of row validation that checks for relationships between columns in one or more table rows. Business rules are enforced by the table filer. While it is possible to have a business rule that only references a single column value, checks of that type are typically performed as a column validation step in the SQL statement.
Column validation	Validation tests that can be applied to the discrete column value, without referencing any other column values. These include the NOT NULL (required) attribute, and either domain or data type validation. Domain and data type validation include correct format, maximum length, and comparisons to constants.
Concurrency	The system's ability to manage more than one concurrent transaction without database corruption. Within SQL, concurrent transactions each have an *isolation level* that determines the allowable interaction between transactions.
Database integrity	Ensures that the database contains valid data, and that the correct relationships between different rows and tables are maintained.
Referential integrity	A type of row validation used on row delete to ensure that the delete does not leave behind foreign keys (pointers) to the deleted row.

Confidential | ©2023

Term	Definition
Row locks	A semaphore that provides concurrency protection for a particular row in a particular table. Row locks may be either *exclusive* (WRITE lock) or *non-exclusive* (READ lock). WRITE locks prevent two concurrent transactions from updating the same row. READ locks prevent transactions from modifying a row that has been read by a different transaction.
Row validation	Any validation test that references two or more column values. This includes unique indices, business rules, and referential integrity.
Table filer	An M routine that applies the changes from INSERT, UPDATE, and DELETE statements to the M global database.
Transaction	A group of one or more SQL statements that reference or modify the database.
Unique indices	A type of row validation that ensures a computed value, composed of the first *N* keys of an index, is unique within the table.

Read & Write Locks

Before an SQL statement can reference or update rows in the database, it must *first* perform the appropriate row locks.

Created tables use a default locking scheme. You must enter the row lock code for mapped tables using the MAP EXISTING GLOBALS option. Any row lock code you enter should be consistent with the locking strategies that are used by your existing applications.

There are two types of locks, *exclusive* WRITE locks and *non-exclusive* READ locks.

WRITE locks are usually implemented by M incremental locks (LOCK +^global_name). SQL uses WRITE locks to ensure that only one transaction can modify a particular row. WRITE locks are always used *prior* to modifying table rows.

READ locks ensure that retrieved rows are not in the process of being updated. It is possible your M applications may not use READ locks or checks. If this is the case, then you should not enter READ lock code for those tables. READ locks are used if the transaction's isolation level is other than READ UNCOMMITTED.

SQL Statements

SELECT Statement

The SELECT statement reads data. If the isolation level is anything other than READ UNCOMMITTED, the SELECTed row performs a READ lock to prevent *dirty* reads and other concurrency violations. Any persistent components of a READ lock are removed only after a COMMIT or ROLLBACK.

INSERT Statement

This statement adds new rows to a table. The M routine that performs the INSERT automatically checks that all required columns have non- null values and performs any domain or data type validation logic to ensure acceptable values.

Unlike the UPDATE and DELETE statements, the INSERT routine *does not* perform a WRITE lock because some tables have calculated primary keys. That computation and the subsequent WRITE lock are deferred to the INSERT code in the table filer.

If the isolation level is anything other than READ UNCOMMITTED, and the statement references any other rows, the statement attempts to perform READ locks on the referenced rows. Any persistent components of any WRITE or READ locks are removed only after a COMMIT or ROLLBACK.

UPDATE Statement

This statement changes column values in table rows. The M routine that performs the UPDATE automatically checks that all required columns have non-null values, and performs any domain or data type validation logic to ensure acceptable values.

The UPDATE statement *does not* allow changes to the primary key columns and performs a WRITE lock *prior* to invoking the update table filer.

If the isolation level is anything other than READ UNCOMMITTED, and the statement references any other rows, the statement attempts to perform READ locks on the referenced rows. Any persistent components of any WRITE or READ locks are removed only after a COMMIT or ROLLBACK.

DELETE Statement

This statement removes rows from a table. The M routine that performs the DELETE, performs a WRITE lock prior to invoking the delete table filer.

If the isolation level is anything other than READ UNCOMMITTED, and the statement references any other rows, the statement attempts to perform READ locks on the referenced rows. Any persistent components of any WRITE or READ locks are removed only after a COMMIT or ROLLBACK.

COMMIT & ROLLBACK Statement

Since the table filers use a *file-as-you-go* approach, the COMMIT statement simply performs any necessary row unlocks. However, the ROLLBACK statement must undo all table row changes. The ROLLBACK operation inverts the previous processed statements, effectively using the table filers *in reverse*.

For example, to reverse an INSERT, a DELETE occurs; to reverse a DELETE, an INSERT is performed; to reverse an UPDATE, another UPDATE takes place, thus changing the columns back to their initial values. After all of the changes have been reversed, the ROLLBACK statement unlocks all rows.

Automatic Table Filers

Table filer routines are automatically generated for tables produced using the CREATE TABLE statement (*after* the appropriate statement reference). The CREATE TABLE statement typically occurs in a sequence such as the following.

```
create table sql_test.created_employees
      (emp_ssn primary character (11),
      name character (15),
      salary numeric (7,2)
      manager character (11))

create index created_employees_by_name for
      created_employees (name)
```

The resulting M global structure for this table consists of a *global name* (derived from the schema definition), followed by a *subscript* (the table ID from the DATA_DICTIONARY.TABLE table), followed in turn by any primary key columns. Each column value is stored vertically, with an integer subscript.

The first time our created table is referenced in an INSERT, UPDATE, or DELETE statement, a table filer is automatically generated. Three corresponding external entry points to the table filer, 'I', 'U', and 'D', respectively, exist in the created table's associated TABLE PRINT report shown on the next page.

In this report, notice also a number of other descriptors for the created table including the table filer routine name 'XX78' (this name is derived from the value entered in the *Filer base routine* prompt in the Schema Information window); the M global name '^SQLT'; the table ID '771'; the primary key column EMP_SSN; and the subscripts 2, 3, and 4 which correspond to the NAME, SALARY, and MANAGER columns, respectively, of the M global structure.

```
Site: KB Systems, Inc.    Schema: SQL_TEST
Table definition, printed on 05/24/95 at 4:32 PM

CREATED_EMPLOYEES - 771

LOGICAL

   Primary key: EMP_SSN
   Columns:
      EMP_SSN                 CHARACTER(11) NOT NULL
      MANAGER                 CHARACTER(11)
      NAME                    CHARACTER(15)
      SALARY                  NUMERIC(7,2)

PHYSICAL
      INSERT filer execute: D I^XX78
      UPDATE filer execute: D U^XX78
      DELETE filer execute: D D^XX78

Primary key 1: EMP_SSN  Average distinct: 10000

^SQLT(771,EMP_SSN            ,2) = NAME   (cs=2)
                            ,3) = SALARY (cs=3)
                            ,4) = MANAGER(cs=4)
```

```
INDICES

CREATED_EMPLOYEES_BY_NAME - 772
^SQLT(772,NAME,EMP_SSN)
Primary key 1: NAME    Average distinct: 100
Primary key 2: EMP_SSN Average distinct: 100
```

Statement Action

The compiled SQL statements use the table filer to save any database changes. The SQL statements also use tags in the utility routine SQL0E to perform row locks and unlocks. The WRITE^SQL0E tag always performs an *exclusive* WRITE lock. The READ^SQL0E tag performs an action appropriate to the transaction's isolation level. If the level is READ UNCOMMITTED, the READ^SQL0E simply quits. If the transaction is READ COMMITTED, the READ^SQL0E tag checks to ensure that no other transaction has a WRITE lock on the specified row.

Filer Action

The table filer communicates with the SQL statement using the

^SQLJ(SQL(1),99,SQLTCTR) global array. The first subscript of this global is the *connection handle*, which uniquely identifies the connection. The second subscript, '99', isolates the table filer information from other connection-related information. The third subscript, 'SQLTCTR' , is a counter which identifies a particular row.

If an SQL statement modifies more than one row, the table filer is called once for each row, and each time it has a different SQLTCTR value. A complete description of the ^SQLJ global array and related variables is provided later in this chapter.

The table filer for the created table begins on the next page.

```
XX78  ;Table filer for 771 [V3.0];05/24/95@4:25 PM
      ; filer for SQL_TEST.CREATED_EMPLOYEES

D     ; delete
      N C,D,K,O,X
      ; check pkeys
      S SQLROWID=$P(^SQLJ(SQL(1),99,SQLTCTR),"~",2,999),K(1)=SQLROWID I '$D(SQLTLEV) S
      SQLTLEV=0
      S (X,SQLTLEV)=SQLTLEV+1 K X(X) S (X,SQLTLEV)=SQLTLEV-1
      ;kill data
      S (C(0),^SQLJ(SQL(1),99,SQLTCTR,0,0))="",D=$G(^SQLT(771,K(1),2))
      I D'="" S $E(C(0),2)=1,O(2)=D,^SQLJ(SQL(1),99,SQLTCTR,-2)=O(2) S
      D=$G(^SQLT(771,K(1),3))
      I D'="" S $E(C(0),3)=1,^SQLJ(SQL(1),99,SQLTCTR,-3)=D S D=$G(^SQLT(771,K(1),4))
      I D'="" S $E(C(0),4)=1,^SQLJ(SQL(1),99,SQLTCTR,-4)=D S
      ^SQLJ(SQL(1),99,SQLTCTR,0,0)=C(0) K ^SQLT(771,K(1))
      ; kill indices
      I $E(C(0),2) K ^SQLT(772,O(2),K(1)) Q

I     ; insert
      N C,D,F,K,N S SQLTBL=771
      ; check pkeys
      S SQLROWID=$P(^SQLJ(SQL(1),99,SQLTCTR),"~",2,999),K(1)=SQLROWID I K(1)="" S SQLERR=583 D
      ER^SQLV3 Q
      I '$D(^SQLT(771,K(1))) G 1
      K ^SQLJ(SQL(1),99,SQLTCTR) S SQLERR=43 D ER^SQLV3 G 4
1     D WRITE^SQL0E
```

```
        I SQLCODE<0 K ^SQLJ(SQL(1),99,SQLTCTR) Q S D=""
        ; load change array
        S C(0)=^SQLJ(SQL(1),99,SQLTCTR,0,0)
        ; insert data S F=0
        I $E(C(0),2) S N(2)=^SQLJ(SQL(1),99,SQLTCTR,2),^SQLT(771,K(1),2)= N(2),F=1
        I $E(C(0),3) S ^SQLT(771,K(1),3)=^SQLJ(SQL(1),99,SQLTCTR,3),F=1
        I $E(C(0),4) S ^SQLT(771,K(1),4)=^SQLJ(SQL(1),99,SQLTCTR,4),F=1 I 'F S
        ^SQLT(771,K(1))=""
        ; set indices
        I $E(C(0),2) S ^SQLT(772,N(2),K(1))="" Q

U       ; update
        N C,D,K,N,O
        ; check pkeys
        S SQLROWID=$P(^SQLJ(SQL(1),99,SQLTCTR),"~",2,999),K(1)=SQLROWID

        ; load change array
        S C(0)=^SQLJ(SQL(1),99,SQLTCTR,0,0)
        ; insert data S F=0
        I $E(C(0),2) S O(2)=$G(^SQLJ(SQL(1),99,SQLTCTR,-2)),N(2)=$G(^SQLJ(
             SQL(1),99,SQLTCTR,2)) S:N(2)'="" ^SQLT(771,K(1),2)=N(2) K:N( 2)="" ^SQLT(771,K(1),2)
        I $E(C(0),3) S N=$G(^SQLJ(SQL(1),99,SQLTCTR,3)) S:N'="" ^SQLT(771, K(1),3)=N K:N=""
              ^SQLT(771,K(1),3)
        I $E(C(0),4) S N=$G(^SQLJ(SQL(1),99,SQLTCTR,4)) S:N'="" ^SQLT(771, K(1),4)=N K:N=""
              ^SQLT(771,K(1),4)
        I '$D(^SQLT(771,K(1))) S ^SQLT(771,K(1))=""
        ; update indices I '$E(C(0),2) Q I
        O(2)="" G 2
        K ^SQLT(772,O(2),K(1)) 2  I N(2)="" Q
        S ^SQLT(772,N(2),K(1))=""
3       Q

4       D ARBACK^SQL0E Q
```

Manual Table Filers

The design of the table filer is geared specifically for SQL. As such, the table filer must support SQL statements and ensure database integrity.

Let's examine these design issues more closely.

SQL Statements

The SQL grammar supports six statements that are relevant to the process: SELECT, INSERT, UPDATE, DELETE, COMMIT, and ROLLBACK. Any number of rows may be processed by each of the first four statements. The SELECT statement reads data; the INSERT, UPDATE, and DELETE statements change rows in the database; and the COMMIT and ROLLBACK statements finalize or discard changes already made by the INSERT, UPDATE, and DELETE statements. In addition, COMMIT and ROLLBACK free any READ or WRITE locks.

Database Integrity

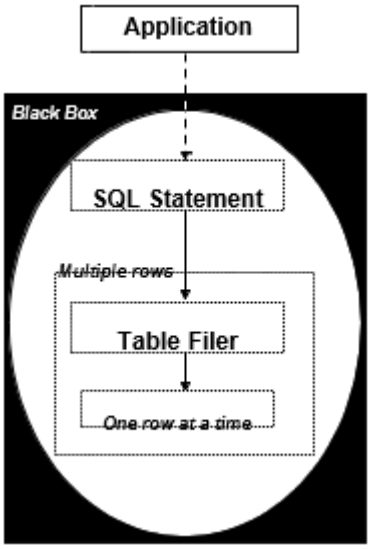
There are two major components to database integrity: data validation and concurrency. Data validation includes both column level constraints and table row level constraints. Column level constraints include data type or domain validation logic and required column checks. The column level constraints are enforced on specific columns by the INSERT and UPDATE statements *prior* to

calling the table filer. Table row level constraints, which include unique indices, referential integrity, and business rules, must be enforced by the table filer.

Most of the concurrency checks are performed by the SQL statements *prior* to executing the table filer logic. However, the INSERT filer logic is responsible for performing a WRITE lock on the new row and READ locks may be performed if additional rows are referenced by the table filer.

Error Handling

Integral to the design process is the provision for error handling. When viewed from the SQL perspective, each statement may process many rows, and either completes successfully for all rows or fails and has no effect on any row in the database.



However, from the perspective of the table filer routine, each statement is viewed as a sequence of single row actions, rather than a single multi-row action. If any row action fails, the filer must ROLLBACK any changes made to the current row and quit with SQLCODE=-1. Control is returned to the statement level. At that time, the compiled SQL statement manages the ROLLBACK of any rows that were processed prior to the failure.

Development Steps

Three steps are necessary to provide SQL modification of a mapped table:

- Add additional domain logic;
- Enter additional table information in the MAP EXISTING GLOBALS option;
- Write the table filer routine.

Step 1: Add Domain Logic

Each column in a mapped table is linked to a *domain*. Each domain is linked to a base *data type* (CHARACTER, DATE, FLAG, INTEGER, MOMENT, NUMERIC or TIME). Each data type provides basic EXTERNAL TO BASE conversion and VALIDATION logic.

If this default logic is sufficient, then you can skip this step. However, if you want the SQL statements to support additional, or different, conversion or VALIDATION logic for a particular domain, you must enter that logic using the DOMAIN EDIT option (DATA DICTIONARY/DOMAIN EDIT). The prompt *Override data type logic?* in DOMAIN EDIT is used to access the EXTERNAL TO BASE and VALIDATION execute logic. Both of these executes use the variable X for input and output.

The EXTERNAL TO BASE execute expects X to be an external value, in a format that the user might enter. This execute should convert X into the base format for the domain's data type. If the external value is in an invalid format, the execute should kill the variable X to indicate an error has occurred. While you may be able to perform the conversion using a single string of M code, we recommend you create a routine to perform the conversion and use the execute to DO the appropriate routine.

The VALIDATION execute expects X to be in base format. The execute then makes any additional checks that are necessary to ensure the value is appropriate. For example, the validation might check that a date value is not in the future by using the code below:

```
I X>+$H K X
```

Step 2: Add Table Information

The MAP EXISTING GLOBALS option contains several prompts that are related to row locking and table filers.

- The TABLE INFORMATION option contains prompts for primary key delimiter, READ/WRITE lock and unlock logic, and table filer executes;
- The COLUMN INFORMATION option allows for the assignment of the change sequence number;
- The PRIMARY KEYS option allows for the Computer Key on the Insert execute logic for generated primary key values;
- And the INDEX option contains information on unique indices.

This section focuses on how values supplied in the prompts are applied by a table filer. For information regarding the location and format of the prompts within the MAP EXISTING GLOBALS option, refer to Chapter 3: Creating Your Data Dictionary.

TABLE INFORMATION Option

To preface our coverage of this option, note that READ and WRITE lock functions provide the following inputs:

Variable	Description
SQLTBL	Table number (from ^SQL(4) table).
SQLROWID	Table row primary key (delimited).

Both the READ and WRITE locks use the variable SQLROWID to represent the composite of *all* primary key columns. If the primary key is composed of more than one column, you may need to enter a primary key delimiter value at the prompt in the Table Features window.

This value should be an ASCII code representing a character that *does not* occur within any of the primary key component columns. The primary key delimiter separates the various primary key columns in the SQLROWID variable.

If you do not enter a primary key delimiter, the tab character (ASCII code = 9) is used by default. The *Allow updates?* prompt determines if this table may be updated. If you enter NO, you are still allowed to enter READ lock logic. If you enter YES, you must enter WRITE lock/unlock and INSERT, UPDATE, and DELETE FILER logic in the sequential Update Table Logic window.

The WRITE lock code sets an exclusive, persistent lock on the table SQLTBL and row SQLROWID. The WRITE unlock code clears a previously established WRITE lock.

The use of the READ lock depends on the transaction's isolation level and your current application's code. If the transaction's isolation level is READ UNCOMMITTED, the READ lock code is not executed. If the level is READ COMMITTED, the code should check to ensure that no other transaction has a WRITE lock on the specific row. The READ unlock resets any persistent READ locks.

***Note**: For the READ COMMITTED level, there is no requirement that READ locks are persistent; simply checking that the row is not currently in use for update is sufficient.*

If your existing applications do not use READ locks, you may choose to skip the READ lock logic. If the READ lock only performs a check, and does not leave a persistent lock, the READ unlock may be skipped.

If the lock functions encounter an error, they must return SQLCODE=-1 and SQLERR equal to a textual error message. In the event of an error, the filer routine must be designed to rollback the current row and quit with SQLCODE=-1.

Insert, Update & Delete Executes

The format of these executes depends on how your table filers work. Typically, these executes contain a DO to a tag in the table filer routine.

COLUMN INFORMATION Option

Each *real* column that may be modified using SQL must have a change sequence number. The change sequence is an integer value that uniquely identifies the column within the table. It is an alternative identifier that is shorter than the column name. These values should be assigned as consecutive positive integers starting with one (1). Gaps between change sequence values should be avoided if possible.

PRIMARY KEYS Option

The Compute Key on Insert code under the INSERT KEY option should only be used for primary key columns that are automatically generated. The ideal code to enter for this execute is a DO to your existing application's code

INDEX Option

The *Number of unique keys* prompt is asked for each index. Since the primary key of the base table is always unique, and since each index contains the complete primary key of the base table, each index row is always guaranteed to be unique. However, certain indices are used to enforce a unique constraint when based on only some of the index keys. If the index is used to support a unique constraint, you should enter the number of keys that comprise the unique portion of the index.

For example, if the EMP_BY_NAME index for the employees is used to ensure that each name is unique, then you would enter '1', indicating that the first key column (NAME) must be unique. If an index existed by MANAGER and NAME, and names were only required to be unique within a

particular manager, you would enter '2', indicating that both the manager and name together must be unique.

Step 3: Writing the Table Filer

The next three statements can be entered by accessing their corresponding options in the Update Table Logic window.

INSERT FILER

The INSERT FILER statement in the Update Table Logic window adds new rows to a table. The INSERT FILER statement establishes the following entries in the ^SQLJ global.

```
^SQLJ(SQL(1),99,SQLTCTR)="I"_SQLTBL
^SQLJ(SQL(1),99,SQLTCTR,0,change_node)=change_flag_string
^SQLJ(SQL(1),99,SQLTCTR,column_sequence)=new_value (if not null)
```

The INSERT FILER must:

- Perform any business rules
- Computer the primary key (if necessary)
- Save the primary key as the second tilde (“~”) piece of the ^SQLJ(SQL(1),99,SQLTCTR) global to support ROLLBACK
- WRITE lock the row
- Check unique indices
- Copy new_values to the database
- Set up any indices

UPDATE FILER

This statement changes column values in table rows. The UPDATE FILER statement establishes the following entries in the ^SQLJ global:

```
^SQLJ(SQL(1),99,SQLTCTR)="U"_SQLTBL_"~"_SQLROWID
^SQLJ(SQL(1),99,SQLTCTR,0,change_node)=change_flag_string
^SQLJ(SQL(1),99,SQLTCTR,-column_sequence)=old_value (if not null)
^SQLJ(SQL(1),99,SQLTCTR,column_sequence)=new_value (if not null)
```

The UPDATE FILER must:

- Perform any business rules
- Check unique indices
- Perform column updates on the database
- Update any indices

DELETE FILER

The DELETE FILER statement removes rows from a table and establishes the following entries in the ^SQLJ global:

```
^SQLJ(SQL(1),99,SQLTCTR)="D"_SQLTBL_"~"_SQLROWID
```


Confidential | ©2023

The DELETE FILER must:

- Perform any business rules
- Copy old_values to the ^SQLJ(SQL(1),99,SQLTCTR,-column_seq) global
- Set up the ^SQLJ(SQL(1),99,SQLTCTR,change_node)=change_flag_string global
- Delete the row from the database
- Delete any indices

Data Structures

The following information is available to the table filer:

Variable	Description
SQL(1)	The unique connection identifier (positive integer).
SQLTCTR	The transaction sequence counter (negative integer). This value is initialized to -1, and is decremented by one for each row processed by INSERT, UPDATE, or DELETE.
$P(SQL(1,1),"~",11)	The isolation level (0=READ UNCOMMITTED, 1=READ COMMITTED).
^SQLJ(SQL(1),99,SQLTCTR)	{D,I,U}_SQLTBL [_"~"_SQLROWID] SQLTBL is the table row id from the TABLE table ^SQL(4). SQLROWID is the table row primary key. If the table has two or more primary key columns, this value is a delimited string. You can specify which delimiter character to use in the MAP EXISTING GLOBALS option.
^SQLJ(SQL(1),99,SQLTCTR, -column_seq)	old_value (base format). This node is not defined if the original value is null. The column_seq is a unique integer value for each column within a specific table.
^SQLJ(SQL(1),99,SQLTCTR, 0,change_node)	change_flag_string. This is a fixed length string of 0 and 1 values, with a maximum length of the number of columns in the table or 200, whichever is less. There is one position for each column_seq. Each position may have a 0, indicating the column has *not* changed, or a 1, indicating the column value has changed. The change_node is an integer value that groups the changed columns as a function of the column_seq. The first value is 0, followed by additional positive integers if the table contains more than 200 columns.
^SQLJ(SQL(1),99,SQLTCTR, column_seq)	new_value (base format). This node is not defined if the new value is null. The column_seq is a unique integer value for each column within a specific table.

Confidential | ©2023

Sample SQL_TEST.EMPLOYEES Table Report

The sample printout below shows the SQL_TEST.EMPLOYEES table definition. This table has a single primary key (EMP_SSN), three additional data values (NAME, MANAGER, and SALARY), and an index by name.

Logic has been added using the MAP EXISTING GLOBALS option to both table filer and WRITE lock prompts. Each of the three data values has a change sequence value indicated by the (cs=N) displayed after the column name in the PHYSICAL section of the report. The DATA column is a composite value that cannot be directly edited, and therefore has no change sequence value.

There is no method for calculating the primary key for this table, but if such code existed, it would be listed along with the other primary key information.

```
Site: KB Systems, Inc.    Schema: SQL_TEST
Table definition, printed on 05/22/95 at 11:39 AM

EMPLOYEES - 10002
         This is a table of all employees.

LOGICAL

   Primary key: EMP_SSN
   Columns:
      DATA                    CHARACTER(50)
            This is a node of employee data.
      EMP_SSN                 CHARACTER(11) NOT NULL
            This is the employee's social security number.
      MANAGER                 CHARACTER(11)
This is the SSN of the employee's manager.

      MANAGER_LINK            EMPLOYEES_ID
            This is a link to the employees' table. Foreign key (MANAGER) to
            EMPLOYEES
      NAME                          CHARACTER(15) NOT NULL
                        This is the employee's name.
      SALARY                        NUMERIC(5,2)
            This is the employee's hourly salary.

PHYSICAL
      INSERT filer execute: D I^TF10002
      UPDATE filer execute: D U^TF10002
      DELETE filer execute: D D^TF10002
         READ lock: L +^SQLEMP(SQLROWID):0 S:'$T SQLERR="Unable to
                        access",SQLCODE=-1 L -^SQLEMP(SQLROWID)
         WRITE lock: L +^SQLEMP(SQLROWID):0 E   S SQLERR="Unable to
                        lock",SQLCODE=-1
         WRITE unlock: L -^SQLEMP(SQLROWID)

Primary key 1: EMP_SSN  Average distinct: 8
                          End at value: "999-00-0000"

^SQLEMP(EMP_SSN) = DATA
                       ";",1)              NAME   (cs=2)
                       ";",2)              SALARY   (cs=4)
                       ";",3)              MANAGER   (cs=5)
```


Confidential | ©2023

```
INDICES

      EMP_BY_NAME - 609
                              employees by name index

     ^SQLEMPN(NAME,EMP_SSN)

Primary key 1: NAME     Average distinct: 8
Primary key 2: EMP_SSN  Average distinct: 1.00
```

Sample Table Filer Routine

This table filer contains separate entry points for DELETE (D^TF10002), INSERT (I^TF10002), and UPDATE (U^TF10002). It is specific to the EMPLOYEES table, which has a single primary key.

The DELETE code loads the primary key into the variable K(1), saves the old values and the change flag string in the ^SQLJ global, and then DELETEs the row and index entries.

The INSERT code loads the primary key into the variable K(1), checks for both a null primary key and duplicate entries, locks the table, loads the change flag string, sets up the data node D, saves the row, and creates the index entry.

The UPDATE code loads the primary key and change flag string, saves any changed values, and resets the index if necessary.

```
TF10002 ;Table filer for SQL_TEST.EMPLOYEES (10002);10:33 AM   22 May 1995

       delete
D      N C,D,K,O,X
       ;check pkeys
       S K(1)=$P(^SQLJ(SQL(1),99,SQLTCTR),"~",2,999)
       save old data
       S C(0)="",D=$G(^SQLEMP(K(1)))
       I $P(D,";",1)'="" S
$E(C(0),2)=1,O(2)=$P(D,";",1),^SQLJ(SQL(1),99, SQLTCTR,-2)=O(2)
       I $P(D,";",2)'="" S
$E(C(0),4)=1,^SQLJ(SQL(1),99,SQLTCTR,-4)=$P(D, ";",2)
       I $P(D,";",3)'="" S
$E(C(0),5)=1,^SQLJ(SQL(1),99,SQLTCTR,-5)=$P(D, ";",3)
       S ^SQLJ(SQL(1),99,SQLTCTR,0,0)=C(0)
       ; kill data
       K ^SQLEMP(K(1))
       ; kill indices
I $E(C(0),2) K ^SQLEMPN(O(2),K(1))
Q
       ;
       ; insert
I      N C,D,F,K,N
       ; check pkeys
       S K(1)=$P(^SQLJ(SQL(1),99,SQLTCTR),"~",2,999)
       I K(1)="" S SQLERR="Missing primary key" G ERROR I '$D(^SQLEMP(K(1))) G 2
       K ^SQLJ(SQL(1),99,SQLTCTR)
       S SQLERR="Duplicate primary key entry exists" G ERROR
2      L +^SQLEMP(K(1)):0 E  S SQLERR="Unable to lock" G ERROR
       ; load change array
       S C(0)=^SQLJ(SQL(1),99,SQLTCTR,0,0)
       ; insert data S D=""
       I $E(C(0),2) S $P(D,";",1)=^SQLJ(SQL(1),99,SQLTCTR,2)
```

```
       I $E(C(0),4) S $P(D,";",2)=^SQLJ(SQL(1),99,SQLTCTR,4)
       I $E(C(0),5) S $P(D,";",3)=^SQLJ(SQL(1),99,SQLTCTR,5)
       I $TR(D,";")="" S ^SQLEMP(K(1))="" Q S ^SQLEMP(K(1))=D
       ; set indices
       I $E(C(0),2) S ^SQLEMPN($P(D,";",1),K(1))="" Q
       ;
       ; update
U      N C,D,K,N,O
       ; load pkeys
       S K(1)=$P(^SQLJ(SQL(1),99,SQLTCTR),"~",2,999)
       ; load change array
       S C(0)=^SQLJ(SQL(1),99,SQLTCTR,0,0)
       ; insert data
       S D=^SQLEMP(K(1)) I $E(C(0),2) S
O(2)=$G(^SQLJ(SQL(1),99,SQLTCTR,-2)),N(2)=$G(^SQLJ(         SQL(1),
99,SQLTCTR,2)),$P(D,";",1)=N(2)
       I $E(C(0),4) S $P(D,";",2)=$G(^SQLJ(SQL(1),99,SQLTCTR,4))
       I $E(C(0),5) S $P(D,";",3)=$G(^SQLJ(SQL(1),99,SQLTCTR,5)) I $TR(D,";")="" S
       ^SQLEMP(K(1))="" Q
       S ^SQLEMP(K(1))=D
       ; update indices I '$E(C(0),2) Q
       I O(2)'="" K ^SQLEMPN(O(2),K(1))
       I N(2)'="" S ^SQLEMPN(N(2),K(1))=""
6      Q
       ;
ERROR  S SQLCODE=-1 K ^SQLJ(SQL(1),99,SQLTCTR) Q
```

Option For Creating Table Filers

The MAP EXISTING GLOBALS option allows a separate table filer execute for INSERT, UPDATE, and DELETE statements. As we have shown, the table filers that are automatically generated for created tables use a separate entry point for each statement type. However, you may choose to create a single entry point that handles all statement types based on the information provided in the

^SQLJ(SQL(1),99,SQLTCTR) global.

If you already use a database management system other than KB_SQL, or if your database structure is very consistent, you may be able to create a single interpreted filer that can handle more than one table.

You could realize consistency and support benefits to such an approach.

If you already have an existing database management system in which you have confidence and that you must continue to support, layering the SQL filer on top of the existing system may reduce development and support time.

For example, if your applications are FileMan-based, it may be possible to build a filer that converts the information provided by the SQL statements into the structures required by *silent* FileMan. While this may reduce system performance, it ensures that both the existing applications and SQL statements perform the same logic, even after a database upgrade.

Another alternative to writing your own table filer from scratch, is to use the same utility that created tables use to generate a table filer (TABLE^SQL0S). Although this utility is only intended to support created tables, it may give you a useful starting point for a custom table filer. You will certainly need to change or enhance this generated filer to match your application's needs, but it can be a handy template for a working table filer.

The TABLE^SQL0S utility requires the following inputs:

Variable	Description
SQLSRC	Table number from ^SQL(4) or 'schema_name.table_name'.
SQLRTN	M routine for table filer.
SQLDTYPE	Valid device type name.
SQLUSER	Valid password.

The following code samples generate a table filer for the PROJECTS and EMPLOYEES table:

```
; compile PROJECTS (K4=10000)
K
S SQLSCR=10000,SQLRTN="XXPROJ"
S SQLDTYPE="DEFAULT",SQLUSER="SHARK"
D TABLE^SQL0S I SQLCODE<0 W !,"Unable to compile!"

; compile EMPLOYEES
K
S SQLSCR="SQL_TABLE.EMPLOYEES",SQLRTN="XXEMP"
S SQLDTYPE="DEFAULT",SQLUSER="SHARK"
D TABLE^SQL0S I SQLCODE<0 W !,"Unable to compile!”
```

Chapter 5: The DDL Interface

KB_SQL Version 3 offers greater levels of flexibility and ease of use by providing a tool to help automate data dictionary loading from other M data dictionaries. This release provides a Data Definition Language (DDL) interface designed to import and export complex DDL commands that we refer to as *script files.*

In this chapter, we overview the Import DDL syntax and extensions that the interface supports, followed by a discussion of how the Import and Export DDL interfaces operate. An alphabetized language reference section follows detailing statement structure and related syntactical components.

The Import DDL Interface

This Import DDL interface provides an alternative to the interactive DBA option MAP EXISTING GLOBALS procedure. The Import DDL interface processes a script file that may be either a global or a host file. Host files are particularly convenient since they may be edited and printed using a word processor. The script file is subsequently input to the Import DDL interface where DDL commands are processed for the KB_SQL Data Dictionary. The script file technique provides you with a text-based, portable file that insulates your definitions from changes to the internal data dictionary structure. The script file may also be easier to update, more readable, and it is reusable.

Overview

DDL enhancements group object and function in the following manner:

- Tables include filer code, default delimiter information, and table level constraints

- Columns contain virtual column or global address information, access requirements, and column level constraints
- Primary keys provide traversal logic and optimization information

The DDL syntax is based on ANSI standard SQL DDL with significant extensions to support M global structures. The CREATE command creates *and* modifies existing database objects. Use ALTER only to change a database object name. The DROP command continues to remove objects from the data dictionary; however, DROP SCHEMA now supports the cascade of component tables.

Order of Statements

- There is no limit to the number of CREATE, ALTER, and DROP statements you may include in script files. In addition, you may order your statements freely; however, a word of caution is appropriate here.
- While DDL statements may occur in any order, that order impacts resulting action. The KB_SQL engine processes each statement *sequentially*.
- Consider, for example, the statements DROP TABLE A.B. and CREATE TABLE A.B., occurring in that order. Assuming table A.B. exists, the KB_SQL engine executes the DROP command first— dropping table A.B.— and then executes the CREATE command, creating table A.B.
- Alternatively, the statements CREATE TABLE A.B. followed by DROP TABLE A.B. cause the KB_SQL engine to first, create A.B. or *alter* existing A.B., and then remove A.B. You should exercise some care to order your statements logically to produce the result you want in your data dictionary.

On the following page, we provide some additional guidelines for ordering your statements.

KB Systems recommends the following statement sequence:

DROP INDEX	Deletes an old index
DROP TABLE	Deletes an old table
DROP SCHEMA	Deletes an old schema
ALTER INDEX	Renames an existing index
ALTER TABLE	Renames an existing table
ALTER SCHEMA	Renames an existing schema
CREATE SCHEMA	Defines a new schema or updates an existing schema
CREATE TABLE	Defines a new table or updates an existing table
CREATE INDEX	Defines a new index or updates existing index

Your DDL script file should start with all DROP statements to delete any old objects. Then follow with all ALTER statements to change the names of any objects. Finally, all CREATE statements should be listed to create any undefined objects or alter any existing objects.

The DDLI CREATE TABLE statement is more forgiving than the standard SQL statement. To change the definition of an existing table, excluding table name changes, the only statement you need to use is the CREATE statement. The CREATE deletes the old definition (removing all indexes) and creates the specified objects (adding new indexes). CREATE also keeps all privileges and other links intact.

The ALTER statement should be used only for changing the name of an existing object. For example, to change an old table name and load its current definition, use the ALTER followed by the CREATE. This is preferable to using the DROP and CREATE combination because DROP and CREATE *DO NOT* preserve privileges and links.

To completely remove an object, the DROP statement should be used. You may also use a DROP followed by a CREATE to clear the old definition, privileges, and other links before loading the new definition.

WARNING

If a schema is dropped, all tables, indices, and foreign keys associated with the schema are *automatically deleted*.

If a table is dropped, all indices and foreign keys associated with the table are *deleted*.

IMPORTANT: The DDLI requires that you set the SQLUSER

="password " variable. A n error message will display if you do not supply the variable.

Operation

To use the DDL interface, you must create a DDL script file that is either a *global* or *host* file. Regardless of the approach, the DDL must conform to the syntax we describe in this chapter. See the alphabetized reference section that follows. In addition, the DDL must strictly adhere to the following rules:

IMPORTANT:

- If anything follows the m_fragment, there must be a space after the m_fragment.
- ALL SCHEMAS, DOMAINS, OUTPUT FORMATS and KEY FORMATS must be created prior to any reference.

To better understand this process, we begin with a look at the creating and execution of a simple global DDL script file. Then we demonstrate the same procedure using a simple host DDL script file.

Using a Global DDL Script

In order to use a global script file, you must create ^SQLIN. This global must have the following structure:

```
^SQLIN(SQL(1),sequence)=DDL text
```

The first subscript, **SQL(1)**, is a unique identifier. There are no particular restrictions on the format of **SQL(1)**. Since it is a subscript, it should be ten characters or less. Typical values for **SQL(1)** include the M $JOB value or a brief name.

The second subscript, **sequence**, must be a positive integer beginning with one (1) and increasing by one for each additional line.

An example of a simple global script that demonstrates the correct structure is provided on the next page.

Simple global DDL script file†

```
^SQLIN(1,1)="CREATE TABLE SQL_TEST.EMPLOYEES"
^SQLIN(1,2)="COMMENT 'This is a table of all employees.'"
^SQLIN(1,3)="(EMP_SSN CHARACTER(11) NOT NULL PRIMARY
GLOBAL ^SQLEMP( "
^SQLIN(1,4)="COMMENT'This is the employees social security
              number.',"
^SQLIN(1,5)="DATA CHARACTER(50) GLOBAL ) PARENT EMP_SSN
CONCEAL"
^SQLIN(1,6)="COMMENT 'This is a node of employee data.',"
^SQLIN(1,7)="NAME CHARACTER(15) NOT NULL PIECE ";",1)
PARENT DATA"
^SQLIN(1,8)="COMMENT 'This is the employee name.',"
^SQLIN(1,9)="SALARY NUMERIC(5,2) PIECE ";",2) PARENT DATA"
^SQLIN(1,10)="COMMENT 'This is the employees hourly salary.',"
^SQLIN(1,11)="MANAGER CHARACTER(11) PIECE ";",3) PARENT DATA"
^SQLIN(1,12)="COMMENT 'This is the SSN of the employees manager.',"
^SQLIN(1,13)="FOREIGN KEY MANAGER_LINK"
```

†The product of this file is the Employees table used throughout the *KB_SQL Data Dictionary Guide*.

Once you define the ^SQLIN global and SQL(1) variable, you are ready to execute the DDL. To do so, use the following command line:

```
S SQLUSER="USER",SQL(1)=1
D DDLI^SQL
I SQLCODE<0 W !,SQLERR
```

Here we set our unique identifier, **SQL(1)**, to 1 as described on the previous page, initiate the DDL interface, and check for success or failure by testing the variable **SQLCODE**. Success is

indicated by an **SQLCODE** value equal to 0; failure is indicated by an **SQLCODE** value equal to -1, accompanied by the **SQLERR** variable equal to a textual error message.

The actual DDL interface process is comprised internally of two phases, parse and import. During a successful process, these phases occur sequentially. DDL is converted from the script file to a KB_SQL table import global, **^SQLIX(SQLJOB,"TABLE")** , that is immediately imported to the KB_SQL Data Dictionary.

In the case of an error in the parse, processing terminates without sequencing to the import phase. Thus, any corruption to the KB_SQL Data Dictionary is prevented.

Required Variables

The DDLI interface requires that you supply the following variables:

SQLUSER=

This variable is the user's password.

SQLCODE=

This variable returns status information.

Optional Variables

Depending on your situation, you may wish to set some of the optional variables below that are available to you during the DDL execution.

IMPORTANT: If the SQLDEBUG, SQLLOG, or SQLTOTAL variables are set, the last 10 lines of the script file will be echoed back to the user in the event of an import error.

Note: *These options are available to provide information you may need for debugging support either in-house or through your vendor's tech support.*

SQLDEBUG=

To turn on debug mode, set this variable to 1 (on).

In debug mode, the DDLI performs extra actions during the parse step to track progress. Each time the parser encounters an ALTER, CREATE, DROP, or SET statement; it places a bookmark at the DDL script line. For this feature to work, each of these statements should be placed at the beginning of the DDL script line. Each new table-rowid value that is parsed and added to the ^SQLIX import global is also tracked under the bookmark. If an error occurs, the DDLI parser deletes all table-rowid entries added since the last bookmark, leaving the ^SQLIX import global in a valid, although incomplete, state. In addition to the usual error information, the variable SQLLINE is also returned. The SQLLINE variable is composed of the line number where the bookmark was set and the actual text of the line, separated by a space.

If an error occurred in debug mode, the operator has two options, other than abandoning the effort.

1) By setting the variable SQLDEBUG="IMPORT" and repeating the DO DDLI^SQL tag, the operator may import the incomplete ^SQLIX file. The operator could then edit the DDL script file, deleting the portion prior to the line SQLLINE and perform another DDLI pass to import the remainder of the script. This may cause problems, however, if the tables in the first import contain foreign keys to tables in the second part, since the foreign keys cannot be resolved.
2) The operator may fix the DDLI script file and set SQLDEBUG="RESTART" and repeat the DO DDLI^SQL tag. This causes the parser to skip to the line SQLLINE and resume the parse. If additional errors occur, the operator may repeat the process. When restarting the parser, the operator must ensure that the SQL(1) variable has not changed, since the import global and bookmark information are indexed by the original SQL(1) value.

To abandon the DDLI parse, set SQLDEBUG="QUIT" and DO DDLI^SQL. This step will delete the temporary structures and clear your connection handles.

SQLDEV=

You may specify a device identifier that you wish to use a log device. The device identifier must be a valid argument for the M OPEN, USE and CLOSE commands.

SQLLOG=

Specify a value of 1 to print the *entire output* of DDL parsing.

WARNING:
If you elect to use the SQLDEV with SQLLOG=1, you may produce a considerable amount of output.

SQLTOTAL=

This value is the total number of lines in the DDL host file or global. Setting this value displays the line number currently processing. This display is suppressed if the SQLDEV option has been set.

Following are several examples of what you may expect to see using some of these options during a successful and unsuccessful parse.

Parse Succeeds

Now we illustrate what you may expect to see in cases of a successful parse using two of the optional variables.

The command line for the example below is:

Confidential | ©2023

```
S SQLUSER="USER",SQL(1)=1,SQLTOTAL=82
D DDLI^SQL
I SQLCODE<0 W !,SQLERR
```

Success message sent to screen

```
Parsing DDL to build table import file...

82   /    82 lines

Parse complete!

Importing data dictionary information...

Import complete!
```

The command line for the next example is:

```
S SQLUSER="USER",SQL(1)=1,SQDEV=3
D DDLI^SQL
I SQLCODE<0 W !,SQLERR
```

Success message sent to printer

```
Parsing DDL to build table import file...

Parse complete!

Importing data dictionary information...

Import complete!
```

Parse Fails

For the next examples, we have purposely inserted an error in line 11 of the global DDL script. The example below demonstrates the use of SQLTOTAL. The command line reads:

```
S SQLUSER="USER",SQL(1)=1,SQLTOTAL=82
D DDLI^SQL
I SQLCODE W !,SQLERR
```


Confidential | ©2023

Error message sent to screen

```
Parsing DDL to build table import file...

11   /   82 lines

Error at line 11 state 166 token XXX type FRAGMENT
```

The message indicates the line on which the error occurred, and the token identifies the component of the DDL statement that produced the error. The state value and type are values that should be provided to your vendor if you are unable to resolve the error on your own.

A variation on this theme sends the results to a printer, SQLDEV=3. The command line is:

```
S SQLUSER="USER",SQL(1)=1,SQLDEV=3
D DDLI^SQL
I SQLCODE<0 W !,SQLERR
```

Error message sent to printer

```
Parsing DDL to build table import file...

Error at line 11 state 166 token XXX type FRAGMENT
```

Note: *These error messages do not indicate all the possible problems that you could conceivably encounter. Errors may occur elsewhere as well. These optional variables simply isolate errors during the parse process.*

Using a Host DDL Script

The DDL interface procedure we have just described for using a global DDL script remains the same here. The only difference is the method in which you create your host DDL script and the manner in which it is referenced on the command line for DDL execution.

The following example, beginning on this page and continuing through the next pages, depicts the creation of a host DDL script for a FileMan file.

Note*: M_fragments such as GLOBAL and PIECE occurring at the end of lines in the following file are followed by a trailing space.*

```
-----------------------------------------------------------
----------------- KB_SQL V3.0 ---------------------------
---Data Dictionary Language Interface(DDLI)Example-------
----(Note: lines that start with two or more dashes------
-----------------  are ignored)  ------------------------
-----------------------------------------------------------

-----------------CREATE DOMAIN Examples------------------

CREATE DOMAIN FM_DATE AS DATE
  COMMENT 'HANDLE YYYMMDD FILEMAN INTERNAL DATES'
  FROM BASE EXECUTE(S %H={BASE} D 7^%DTC S {INT}=X)
  TO BASE EXECUTE(S X={INT} D H^%DTC S {BASE}=%H)

CREATE DOMAIN FM_MOMENT AS MOMENT
  COMMENT 'INTRO-CONVERT YYYMMDD.HHMMSS AND $H DATES'
  FROM BASE EXECUTE
    (S %H={BASE} D YMD^%DTC S {INT}=X_$S(%:"."_%,1:""))
  TO BASE EXECUTE
    (S X={INT} D H^%DTC S {BASE}=%H_$S(%T:","_%T,1:""))
```

```
CREATE TABLE FMTEST.FM_TABLE DELIMITER
  94 FILEMAN FILE 605498

( FM_TABLE_ID INTEGER(10) NOT NULL GLOBAL ^MICRO(

  ,PRIMARY KEY ( FM_TABLE_ID START AT 0 END IF
  ('{KEY}))

  ,NAME CHARACTER(30) COMMENT 'AUTO GENERATED BY
   FILEMAN' HEADING 'NAME' FILEMAN FILE 605498
   FIELD .01 NOT NULL PARENT FM_TABLE_ID GLOBAL
   ,0) PIECE 1

  ,NUMBER NUMERIC(13,6) HEADING 'NUMERIC
   VALUE' FILEMAN FILE 605498 FIELD 1
   PARENT FM_TABLE_ID GLOBAL,0) PIECE 2

  ,INTEGER_VALUE INTEGER(5) FILEMAN FILE 605498
   FIELD 2 PARENT FM_TABLE_ID GLOBAL ,0) PIECE 3

  ,DATE_VALUE FM_DATE FILEMAN FILE 605498 FIELD 3
   PARENT FM_TABLE_ID GLOBAL ,0) PIECE 4

  ,DATE_TIME_VALUE FM_MOMENT FILEMAN FILE 605498
   FIELD 4 PARENT FM_TABLE_ID GLOBAL ,0) PIECE 5

  ,TIME_VALUE FM_MOMENT FILEMAN FILE 605498
   FIELD 5 PARENT FM_TABLE_ID GLOBAL ,0)
   PIECE 6

  ,VAR_PTR NUMERIC FILEMAN FILE 605498 FIELD 6 PARENT
   FM_TABLE_ID GLOBAL ,0) PIECE 7

)
```


Confidential | ©2023

```
      CREATE INDEX FM_TABLE_X1 FOR
      FMTEST.FM_TABLE  ( NAME GLOBAL
      ^MICRO("B",

        ,FM_TABLE_ID PARENT NAME GLOBAL ,

        ,PRIMARY KEY(NAME,FM_TABLE_ID)

      )

      CREATE TABLE FMTEST.FM_TABLE_LEVEL_2 DELIMITER 94
        FILEMAN FILE 605498.07

      ( FM_TABLE_ID REFERENCES FMTEST.FM_TABLE NOT
         NULL GLOBAL ^MICRO(

        ,FM_TABLE_LEVEL_2_ID INTEGER(10) NOT NULL PARENT
         FM_TABLE_ID GLOBAL ,"L1",

        ,PRIMARY KEY ( FM_TABLE_ID START AT 0 END IF ('{KEY}),
         FM_TABLE_LEVEL_2_ID START AT 0 END IF ('{KEY}) )

        ,LEVEL_2 CHARACTER(20) FILEMAN FILE 605498.07 FIELD
         .01  PARENT FM_TABLE_LEVEL_2_ID GLOBAL ,0)
                                                PIECE 1

      )

      CREATE INDEX FM_TABLE_LEVEL_2_X1 FOR

        FMTEST.FM_TABLE_LEVEL_2
        ( FM_TABLE_ID GLOBAL ^MICRO(
          ,LEVEL_2 PARENT FM_TABLE_ID GLOBAL , "L1","B",
          ,FM_TABLE_LEVEL_2_ID PARENT LEVEL_2 GLOBAL ,
          ,PRIMARY
          KEY(FM_TABLE_ID,LEVEL_2,FM_TABLE_LEVEL_2_ID)
)

----------------------End of DDL File--------------------
```

Once you create your host DDL script file, it may be loaded to DDL interface using the following command lines:

```
S SQLUSER="USER"
S SQLFILE="\DDL_FILE.TXT"
D DDLI^SQL
I SQLCODE<0 W !,SQLERR
```

The previous examples of messages for failed and successful parses can occur here as well. The only difference is that your command line reflects the use of a host DDL script in place of the previous global DDL script.

DDL Commands

Note: *All names (e.g., index_name, domain_name, etc.) must be valid SQL_IDENTIFIERs. An SQL IDENTIFIER is a name, starting with a letter (A-Z), followed by letters, numbers (0-9), or underscores*

'_'. The last character in the name cannot be an underscore. The length of the name must not exceed 30 characters; however, a maximum length of 18 characters is recommended for portability to other relational database systems.

ALTER INDEX

ALTER INDEX [schema_name.]index_name Rename name

ALTER DOMAIN

ALTER DOMAIN domain_name Rename name

ALTER KEY FORMAT

ALTER KEY FORMAT key_format_name Rename name

ALTER OUTPUT FORMAT

```
ALTER OUTPUT FORMAT output_format_name
FOR data_type_name
RENAME name
```

ALTER SCHEMA

ALTER SCHEMA schema_name RENAME name

ALTER TABLE

ALTER TABLE [schema_name.]table_name RENAME name

CREATE DOMAIN

```
CREATE DOMAIN domain_name [AS] data_type_name
[COMMENT literal]
[LENGTH integer [SCALE integer]]
[OUTPUT FORMAT output_format_name]
[FROM BASE {EXECUTE(m_execute) or
          EXPRESSION(m_expression)]
[TO BASE {EXECUTE(m_execute) or
         EXPRESSION(m_expression)]
[NULLS EXIST]
[REVERSE COMPARISONS]
[OVERRIDE COLLATION]
[NO SEARCH OPTIMIZATION]
[NO COLLATION OPTIMIZATION]
```

CREATE INDEX

```
CREATE [UNIQUE] INDEX [schema_name.]index_name
FOR [schema_name.]table_name [COMMENT literal]
(
column_name[address_specification]
```

```
        [,  column_name[address_specification]]...

        [,PRIMARY KEY (column_name primary_key_specification
                [, column_name primary_key_specification]...)]
        [,FOREIGN KEY foreign_key_name
        [COMMENT literal] (column_name [,column_name]...)  table_reference]...
)
```

CREATE KEY FORMAT

```
        CREATE KEY FORMAT key_format_name
        FROM BASE {EXECUTE(m_execute) or
                EXPRESSION(m_expression)}
        [COMMENT literal]
        [EQUIVALENT]
        [NON NULLS EXIST]
        [NULLS EXIST]
        [REVERSE COMPARISONS]
```

CREATE OUTPUT FORMAT

```
        CREATE OUTPUT FORMAT output_format_name
        FOR  data_type_name
        [COMMENT literal]
        TO EXTERNAL {EXECUTE(m_execute) or
                EXPRESSION(m_expression)}
        [[LENGTH] integer]
        [{CENTER or LEFT or RIGHT} [JUSTIFY]]
        [EXAMPLE literal]
```

CREATE SCHEMA

```
        CREATE SCHEMA schema_name
        [COMMENT literal]
        [GLOBAL  m_fragment]
```

CREATE TABLE

```
        CREATE  TABLE [schema_name.]table_name
        [COMMENT literal]
        [DELIMITER integer]

        [[READ LOCK(m_execute) UNLOCK(m_execute)]
        [WRITE LOCK(m_execute) UNLOCK(m_execute)]
        [INSERT(m_execute) COMMIT(m_execute)
                ROLLBACK(m_execute)]
        [UPDATE(m_execute)   COMMIT(m_execute)
```


Confidential | ©2023

```
        ROLLBACK(m_execute)]
[DELETE(m_execute)   COMMIT(m_execute)
        ROLLBACK  (m_execute)]]

[external_file_specification]
(
table_column_specification [, table_column_specification]...

[, PRIMARY KEY(column_name primary_key_specification
        [, column_name primary_key_specification]...)]

[, FOREIGN KEY foreign_key_name [COMMENT literal]
        (column_name [, column_name]...)  table_reference]...

[, [CONSTRAINT constraint_name] CHECK(condition)]...
)
```

DROP DOMAIN

```
DROP DOMAIN domain_name
```

DROP INDEX

```
DROP  INDEX [schema_name.]index_name
```

DROP KEY FORMAT

```
DROP KEY FORMAT key_format_name
```

DROP OUTPUT FORMAT

```
DROP OUTPUT FORMAT output_format_name
FOR  data_type_name
```

DROP SCHEMA

```
DROP SCHEMA schema_name
```

DROP TABLE

```
DROP  TABLE [schema_name.]table_name
```

Syntactical Components

address_specification

```
[PARENT  column_name]
[CHANGE SEQUENCE integer]
[GLOBAL  m_fragment]
[PIECE m_fragment]
[EXTRACT FROM integer TO integer]
```


Confidential | ©2023

Rules
The CHANGE SEQUENCE is ignored for index columns.

If m_fragments are used after the GLOBAL or PIECE key words, they must either be the last token on the line or followed by a space character.

condition

A valid SQL condition that returns true, false or unknown.

domain_specification

{domain_name [(length [,scale])]} or table_reference

external_field_specification

{EXTERNAL FILE literal FIELD literal} or fileman_field_specification

external_file_specification

{EXTERNAL FILE literal} or
{FILEMAN FILE numeric}

Fileman_field_specification

FILENAME FILE numeric FIELD numeric
[CODES literal]
[COMPUTED]

literal

quote_character [any_non_quote or embedded_quote]...
quote_character.

Rule
Any_non_quote is any printable character other than the single
Quote character (‘) and the double quote character (“).
An embedded quote is:
quote_character quote character

m_execute

One or more M commands that may be executed.

m_expression

An M expression that evaluates to a value.

m_fragment

A partial M expression that does not contain embedded spaces other than within literals.

Rule

M_fragments must either be the last token on the line or followed by a space character.

Confidential | ©2023

primary_key_specification

[AVG DISTINCT numeric]
[KEY FORMAT key_format_name]
[INSERT KEY (m_execute)]
[START AT literal]
[END AT literal]
[SKIP SEARCH OPTIMIZATION]
[ALLOW NULL VALUES]
[PRESELECT(m_execute)]
[CALCULATE KEY (m_execute)]
[VALID KEY (m_expression)]
[END IF (m_expression)]
[POSTSELECT (m_execute)]

Rule
KEY FORMAT and ALLOW NULL VALUES may only be used with indices. Similarly, INSERT KEY may only be used with tables.

table_column_specification

column_name
domain_specification
{address_specification or {VIRTUAL(sql_expression)}}
[COMMENT literal]
[HEADING literal]
[OUTPUT FORMAT output_format_name]
[PROGRAMMER ONLY]
[CONCEAL]
[PRIMARY primary_key_specification]
[NOT NULL]
[UNIQUE]
[external_field_specification]

table_reference

REFERENCES [schema_name].table_name
[ON DELETE {NO ACTION or CASCADE or SET DEFAULT or SET NULL}]

Confidential | ©2023

The Export DDL Interface

Existing KB_SQL data dictionary definitions can be exported as a DDLI script. This can be helpful in the mapping process to illustrate how current tables were (or could have been) defined using DDLI statements.

This interface is designed for programmers.

Required Input Variables

SQLUSER=

This variable is the user's password.

SQLDTYPE=

This variable is the device type to be used for the export.

Optional Input Variables

DDLI("SCHEMA")=

This is the SCHEMA (or '*' for all) to be exported.

DDLI("TABLE")=

This is TABLE (or Internal ID) to be exported. If table name is not unique, specify the schema name also.

SchemaName.TableName
SchemaName.*
TableName

DDLI("INDEX")=

IndexTableName
TableName.IndexTableName
TableName.*
Schemaname.TableName.*
SchemaName.*.*

DDLI("OUTPUTFORMAT")=

The output format name to be exported.
OutputFormatName (or '*' for all)

DDLI("KEYFORMAT")=

The key format name to be exported.
KeyFormatName (or '*' for all)

DDLI("TAB")=

The defaults for the tab character string are as follows:

Output to file	**: $C (9)**
Global	**: 4 - Spaces**

DDLI("TO_GLOBAL")=

The default for the global reference is `^SQLX ($JOB,"DDLI")`.

DDLI("TO_FILE")=

Enter the filename to receive the output.

DDLI("SILENT")=

If this variable is set, output messages will not be displayed while working.

DDLI("SINGLE")=

This variable will export only a single definition. When extracting table definitions, the default behavior is to extract all related objects.

Optional Output Variables

DDLI("MSG")=

If this variable is set, an informational message will be displayed after completion of the export.

SQLERR=

This variable displays any error text.

Export DDL Interface Examples

Example #1: Export DDLI for Tables

This example shows how to use the Export DDLI for all tables in the SQL_TEST schema.

```
; Sample DDLI Export
K
S SQLUSER="SHARK"
S SQLDTYPE="DEFAULT"
S DDLI("SCOPE")="SCHEMA"
S DDLI("SCHEMA")="SQL_TEST"
S DDLI("TO_FILE")="C:\TEMP\SQLTEST.DDL"
D EXDDLI^SQL
I $D(SQLERR) W !,"Error:  ",SQLERR Q
I $D(DDLI("MSG")) W !,DDLI("MSG")
Q
```


Confidential | ©2023

Example #2: Exporting the DDLI definition

This example shows how to use the Export DDLI to export just the definition for the EMPLOYEES table.

```
; Sample DDLI Export
K
S SQLUSER="SHARK"
S SQLDTYPE="DEFAULT"
S DDLI("TABLE")="SQL_TEXT.EMPLOYEES"
S DDLI("SINGLE")=""
D EXDDLI^SQL
I $D(SQLERR) W !,"Error: ",SQLERR Q
I $D(DDLI("MSG")) W !,DDLI("MSG")
Q
```

Example #3: Exporting all domain definitions

This example shows how to use the Export DDLI to export all domain definitions.

```
; Sample DDLI Export
K
S SQLUSER="SHARK"
S SQLDTYPE="DEFAULT"
S DDLI("DOMAIN")="*"
D EXDDLI^SQL
I $D(SQLERR) W !,"Error:   ",SQLERR Q
I $D(DDLI("MSG")) W !,DDLI("MSG")
Q
```


Confidential | ©2023